

DATA TRUST_SC: A PRIVACY-PRESERVING AND
BLOCKCHAIN-INSPIRED FRAMEWORK FOR SECURE DATA
SHARING IN SMART CITIES

JAWADUL KARIM (C221010)

ANIK CHOWDHURY (C201089)

MOHAMMED IMRAN (C221018)



INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG (IIUC)

DATA TRUST_SC: A PRIVACY-PRESERVING AND BLOCKCHAIN-INSPIRED
FRAMEWORK FOR SECURE DATA SHARING IN SMART CITIES

ANIK CHOWDHURY (C201089)
JAWADUL KARIM (C221010)
MOHAMMED IMRAN (C221018)



THESIS SUBMITTED IN FULFILMENT FOR THE DEGREE OF
B. SC. IN COMPUTER SCIENCE AND ENGINEERING

SUPERVISED BY
MOHAMMOD SAZID ZAMAN KHAN
ASSISTANT PROFESSOR

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
INTERNATIONAL ISLAMIC UNIVERSITY CHITTAGONG
CHITTAGONG, BANGLADESH

DECLARATION

I/We hereby declare that the work in this thesis is my/our own except for quotations and summaries which have been duly acknowledged.

04 July 2026

JAWADUL KARIM
ID: C221010

ANIK CHOWDHURY
ID: C201089

MOHAMMED IMRAN
ID: C221018

SUPERVISOR'S DECLARATION

I/We hereby declare that we have read this thesis and in my/our opinion this thesis is sufficient in terms of scope and quality for the award of the degree of B. Sc. in Computer Science & Engineering

04 July 2026

MOHAMMOD SAZID ZAMAN KHAN
Assistant Professor
Department of CSE, IIUC

DECLARATION OF THESIS / PROJECT REPORT AND COPYRIGHT

THESIS / PROJECT REPORT TITLE:

DataTrust_SC: A Privacy-Preserving and Blockchain-Inspired Framework for Secure Data Sharing in Smart Cities

AUTHORS:

SL NO.	AUTHOR'S NAME	STUDENT ID	SIGNATURE
1	Jawadul Karim	C221010	
2	Anik Chowdhury	C201089	
3	Mohammed Imran	C221018	
NAME OF SUPERVISOR: Mohammod Sazid Zaman Khan			
SIGNATURE OF SUPERVISOR:			

I/we declare

1. My/ Our thesis to be published as online open access (full text) at IIUC database or archive.

2. Dept. of CSE, IIUC reserves the right as follows:

i. The thesis is the property of Dept. of CSE, IIUC

ii. The Library of IIUC has the right to make copies for the purpose of research only.

iii. The Library has the right to make copies of the thesis for academic exchange.

ACKNOWLEDGEMENT

First and foremost praise be to Almighty Allah for all his blessings for giving me patience and good health throughout the duration of this thesis work.

I am very fortunate to have Professor Mohammad Sazid Zaman Khan as a research supervisor.

Moreover, I am grateful to the faculties and staffs of the Dept. of CSE, IIUC

I would like to thank everyone.

ABSTRACT

The rapid urbanization into smart city phenomena has produced data-rich ecosystems that facilitate the provision of intelligent services while also raising concerns regarding privacy, data integrity, unauthorized access, auditability, and trust among multiple parties. Current centralized data-sharing models often have limitations including: vulnerability to single points of failure, lack of transparency, poor policy enforcement and decreased cooperation among different entities (municipal authorities, companies, researchers, service providers, and citizens). This thesis proposes the DataTrust_SC, a privacy-aware and blockchain-inspired framework for enabling secure and auditable sharing of smart city data.

The DataTrust_SC combines: AES-based dataset encryption, Role and Attribute-Based Access Control (RBAC/ABAC), Hash-based identification of data, Simulated smart-contract-based access authorization and well-structured audit trail logging. In the implementation of the DataTrust_SC prototype, the process of storing and checking authorization policies has been represented through a backend "PolicyContract" component that mimics the execution logic of an actual blockchain smart contract. Hash-chained log records containing 'Previous Hash', 'Hash', 'Index', and 'contractID' fields along with a 'chain validation function ' have been used for tamper-evident logging.

The implementation of the system is a web-based simulation prototype using React front-end, Node.js/Express back-end, and MongoDB database. Features of the system include: dataset registration, storing encrypted datasets, request analysis, authorization decision (grant or reject), creation of an audit trail, displaying data and triggering warnings and alerts (e.g. Disaster warnings). The experiments were conducted using synthetic smart city data simulating characteristics of the city in relation to the environment, traffic, utility services, and civic data. We analyzed: Correctness of authorization, Denied access attempts, Completeness of the logs, Latency indicators of the authorization process simulation, Transaction costs in a simulated sense, Dataset distribution, Disaster monitoring.

The proposed framework successfully managed to enforce stored access policies, prevent unauthorized raw data disclosure and store log entries in a hash-chained manner to provide integrity checking of logs. Furthermore, the disaster monitoring module proves the viability of applying secure governed data for urban safety-related services such as issuing warnings and alerts when some thresholds are crossed. The study can demonstrate that applying cryptography, finer-grained access control policy, and blockchain-inspired audit trail is feasible at the prototype level, while real blockchain consensus mechanisms, fully functional deployed smart contracts with their real gas fees, real smart city datasets with proper security testing, and larger-scale operations validation are still areas for future work.

Keywords

Smart City Data Sharing, Privacy-Preserving Architecture, Attribute-Based Access Control, AES Encryption, Simulated Smart Contract, Hash-Chained Audit Logging, Blockchain-Inspired Auditability, Secure Data Management, Disaster Monitoring System.

TABLE OF CONTENTS

DECLARATION.....	3
SUPERVISOR’S DECLARATION.....	4
DECLARATION OF THESIS / PROJECT REPORT AND COPYRIGHT.....	5
ACKNOWLEDGEMENT.....	7
ABSTRACT.....	8
TABLE OF CONTENTS.....	9
LIST OF TABLES.....	12
LIST OF ILLUSTRATIONS.....	13
LIST OF ABBREVIATIONS.....	14
CHAPTER I.....	15
1.1 Background of Smart Cities.....	15
1.2 Problem Statement.....	17
1.3 Research Motivation.....	19
1.4 Research Objectives.....	21
1.5 Research Questions.....	21
1.6 Scope of the Research.....	22
1.7 Contributions of the Thesis.....	23
1.8 Thesis Organization.....	25
CHAPTER II.....	27
2.1 Introduction to the Literature Review.....	27
2.2 Smart City Data Management.....	28
2.2.1 Data Sources in Smart Cities.....	29
2.2.2 Data Types in Smart Cities.....	30
2.2.3 Smart City Data Lifecycle.....	31
2.3 Privacy Challenges in Smart Cities.....	32
2.3.1 Data Privacy Threats.....	32
2.3.2 Security Attacks.....	33
2.4 Blockchain Fundamentals.....	35
2.4.1 Blockchain Architecture.....	36
2.4.2 Smart Contracts.....	37
2.5 Blockchain for Data Sharing.....	38
2.5.1 Centralized vs Decentralized Sharing.....	39
2.5.2 Existing Blockchain-Oriented Approaches.....	39
2.5.3 DataTrust_SC Positioning and Simulation Boundary.....	40
2.6 Access Control in Smart City Data Sharing.....	41
2.7 Privacy Preserving Techniques.....	42

2.8 Auditability and Hash-Chained Logging.....	43
2.9 Simulation-Based Evaluation and Its Limits.....	44
2.10 Research Gaps.....	44
CHAPTER III.....	48
3.1 System Architecture.....	48
3.2 Threat Model.....	49
3.3 Design Requirements.....	50
3.4 Assumptions.....	51
3.5 Problem Formulation.....	52
CHAPTER IV.....	54
4.1 Overview of DataTrust_SC.....	54
4.2 Architecture Design.....	55
4.2.1 Policy and Audit Layer.....	55
4.2.2 Off-Chain Components.....	55
4.3 Privacy Preserving Mechanism.....	56
4.3.1 Encryption Strategy.....	56
4.3.2 Key Management.....	57
4.3.3 Identity Protection.....	57
4.4 Access Control Model.....	57
4.4.1 Role-Based and Attribute-Based Access.....	57
4.4.2 Policy Enforcement Logic.....	58
4.5 DataTrust_SC framework workflow.....	59
4.6 Simulation Prototype Representation.....	61
4.7 Proposed Framework vs Simulation Prototype.....	63
4.8 Security Features.....	63
CHAPTER V.....	65
5.1 Implementation Environment.....	65
5.1.1 System Architecture Overview.....	66
5.1.2 Software and Tools Used.....	67
5.2 Prototype Functional Implementation.....	67
5.2.1 Dataset Registration and Encryption Module.....	68
5.2.2 Access Policy Enforcement Module.....	68
5.2.3 Simulated Blockchain Audit Logging Module.....	69
5.2.4 Chain Verification Module.....	70
5.2.5 Live Data Simulation Module.....	70
5.2.6 Disaster Warning and Aid Center Module.....	71
5.3 Dataset Description.....	73
5.4 Experimental Scenarios.....	74
5.5 Evaluation Metrics.....	75

CHAPTER VI.....	77
6.1 Access Control and Audit Log Validation.....	77
6.1.1 Successful Authorization Scenario.....	78
6.1.2 Denied Authorization Scenario.....	79
6.2 Audit-Chain Integrity Verification.....	80
6.3 Simulated Performance Analysis.....	82
6.3.1 Simulated Transaction Cost.....	83
6.3.2 Simulated Authorization Latency.....	85
6.4 Dataset Distribution Analysis.....	87
6.5 Disaster Monitoring Module Evaluation.....	87
6.6 Discussion of Results.....	89
CHAPTER VII.....	91
7.1 Conclusion.....	91
7.2 Contributions Revisited.....	92
7.2.1 Conceptual Contributions.....	92
7.2.2 Practical Contributions.....	92
7.3 Limitations.....	93
7.4 Future Research Directions.....	93
REFERENCES.....	95
APPENDIX A.....	98
APPENDIX B.....	99

LIST OF TABLES

Table 2.1: Privacy and Security Challenges in Smart Cities.....	35
Table 2.2: Blockchain Components and Smart Contract Functions.....	38
Table 2.3: Conceptual Comparison of Data-Sharing Approaches.....	40
Table 2.4: Research Gaps and DataTrust_SC Response.....	46
Table 3.1: Threat Model Summary.....	50
Table 3.2: Design Requirements Summary.....	51
Table 4.1: DataTrust_SC Architectural Components.....	56
Table 4.2: Proposed Framework vs Simulation Prototype.....	63
Table 4.3: Security Features of DataTrust_SC.....	64
Table 5.1: Access Control Policy Matrix.....	69
Table 5.2: Dataset Provider and Category Mapping.....	73
Table 5.3: Evaluation Metrics Definition.....	76

LIST OF ILLUSTRATIONS

Figure 3.1: DataTrust_SC Conceptual System Architecture.....	49
Figure 4.1: Proposed DataTrust_SC Conceptual Workflow.....	60
Figure 4.2: Simulation Prototype Workflow.....	62
Figure 5.1: Simulated Prototype.....	66
Figure 5.2: Live Data Simulation Configuration Interface.....	71
Figure 5.3: Live Data Simulation Active Status Interface.....	71
Figure 5.4: Disaster Warning and Aid Center.....	72
Figure 6.1: Successful Access Request Interface & Audit Log.....	78
Figure 6.2: Unauthorized Access Attempt Interface & Audit Log.....	79
Figure 6.3: Audit-Chain Verification Output When Integrity Is Maintained.....	81
Figure 6.4: Audit-Chain Verification Output When Integrity Is Broken.....	82
Figure 6.5: Blockchain Analytics Dashboard Summary Metrics.....	83
Figure 6.6: Simulated Gas Cost Over Time.....	83
Figure 6.7: Simulated Gas Cost Distribution Across Transaction Ranges.....	84
Figure 6.8: Simulated Average Gas Cost By Sector.....	84
Figure 6.9: Authorization Latency Over Time.....	85
Figure 6.10: Average Authorization Latency by Data Provider Type.....	85
Figure 6.11: Relationship Between Simulated Gas Cost and Authorization Latency..	86
Figure 6.12: Data Distribution by Sector.....	87
Figure 6.13: Disaster Warning & Aid Center Overview Dashboard.....	88
Figure 6.14: Example of Critical Alert with Recommended Actions.....	88

LIST OF ABBREVIATIONS

ABAC	Attribute-Based Access Control
AES	Advanced Encryption Standard
ICT	Information & Communication Technology
RBAC	Role-Based Access Control
IoT	Internet of Things
PoW	Proof of Work
PoS	Proof of Stake
API	Application Programming Interface
TPS	Transactions Per Second
TLS	Transport Layer Security
PII	Personally Identifiable Information

CHAPTER I

INTRODUCTION

1.1 BACKGROUND OF SMART CITIES

As urbanization reaches new peaks, cities must develop and manage their services in an effective, sustainable and inclusive manner. With more than 68 percent of the world's population projected to be urban by 2050 (according to the United Nations), cities face numerous challenges, including transportation and energy needs, health and safety demands, and pollution control. Therefore, smart cities are emerging as a new paradigm and the next stage in managing the city, delivering public services, and improving citizens' well-being by leveraging advanced information and communication technologies (ICT), Internet of Things (IoT) devices, cloud computing, data analytics, and intelligent systems. Smart cities are regarded as operating systems where physical infrastructure, ICT resources and human inputs are merged to ensure the non-stop monitoring, evaluation and decision-making processes.

The evolution to smart cities happened through stages. From the 1990s to the early 2000s, smart city initiatives were termed 'digital cities', aiming to boost internet access, spread e-governance and implement basic ICT infrastructure. Later, when communication technologies progressed, with the development of the Internet of Things (IoT), these city projects turned to 'intelligent cities' which focused more on sensors, automation and smart controls. Today, smart cities, representing an advancement of earlier approaches, incorporate IoT, cloud computing, big data analytics, artificial intelligence, and blockchain and aim to create 'smart, autonomous and human-centric' cities (Albino et al., 2015).

Data is the principal resource for smart cities. Vast amounts of varied, evolving data are generated every second through various devices, including IoT sensors, surveillance systems, smart meters, mobile phones, social networks, and licensed services applications. This data will collect real-time information on traffic congestion, air quality, energy consumption and distribution, waste collection, and citizen safety. IoT devices are particularly important as they continuously monitor the physical environment and infrastructure, therefore minimizing the gap between the physical and digital city layers. Affordable sensor devices and efficient wireless communication technologies have significantly amplified the volume and the granularity of urban data collection.

Beyond the machine-generated data, citizen-generated data in smart cities comes from mobile applications, participation sensing platforms and social networks. Citizen information includes not only consumed data, but also contributed data (e.g., reporting an incident, sharing travel information, providing feedback about services). The participatory element improves the awareness of the situations and facilitates a more responsive governing model. However, citizen-generated data raises great concerns of privacy and ethics because this type of data contains valuable personal information, such as mobility tracing, user behavior and social networks (Kitchin, 2014), so citizens' trust and safety of data must be guaranteed in smart city environments.

Data utilization for public services also constitutes an essential feature of smart cities. Data-driven platforms represent a new, important approach to enhance traffic systems, power grids and health services, as well as emergency response. Intelligent transportation systems use traffic data for optimizing traffic flows and mitigating pollution, while intelligent energy systems can use consumption data for adapting power supply and demand dynamically. Data analytics within the health sector help in monitoring disease epidemics, allocating resources effectively to healthcare units and performing telemedicine. These applications provide evidence on data being a strategic resource by which cities can progress from reactive to proactive and predictive service delivery.

Data-driven decision-making forms the bedrock of smart city operations. By leveraging a combination of data collection via numerous sensors and the adoption of modern analytics, city authorities will be able to generate actionable insights that can inform policy development, infrastructure capital investment, and operational planning. By deploying Machine Learning models and predictive analytics, cities will be able to anticipate traffic congestion, predict energy demand, anticipate infrastructure failure and forecast environmental hazards. Such features can boost efficiency, reduce operational costs, and increase the quality of services. However, integrity, availability and reliability of data are essential for the success of data-driven decision making, as inaccurate and manipulated data may lead to sub-optimization or even undesirable decision making (Hashem et al., 2016).

Challenges of data governance, security and privacy are of major concerns to smart cities due to the enormous data transformation capabilities they have. Typical designs of smart cities generally adopt centralized data storage and processing platforms, which are either run by the city authority or by a third-party service provider. Centralized models not only reduce the complexity of data aggregation and analysis but also contribute to a single point of failure and easy targets for cyber-attack. Furthermore, they also bring concerns about the transparency regarding the utilization of the data, sharing of information and access management. Citizens might be concerned that information collected via mobile applications, participatory sensing and social networks is utilized for unintended purposes without authorization or accountability (Kitchin, 2014).

Inter-organizational data sharing is also another complicated aspect in the smart city environment. Urban management should involve multi-stakeholder collaboration where city authorities are expected to work together with other parties involved, which include the government, private organizations that offer services and the community. Various stakeholders will likely possess diverse policies regarding data ownership, access rights and their own trust models. The ability for different entities to securely share data in a privacy-preserving manner and verifiable manner beyond organizational boundaries is the biggest challenge from both a technical and governance perspective. Standard access control mechanisms in practice often fail to offer transparency and scalability in terms of providing a fine-grained authorization framework in a rapidly developing urban context (Lu et al., 2019).

Consequently, there is an increasing awareness of the need to embrace trust and privacy paradigms in the data management of third-generation smart cities. To mitigate some of these problems, new technologies such as blockchain with

conceptual designs such as immutable logging and distributed authorization will have the potential to do so. The proposed approach, with the help of cryptography and privacy-preserving mechanisms, will be capable of revolutionizing data sharing and data governance in smart city environments, and these observations serve as the motivations for this thesis work that proposes DataTrust_SC architecture for privacy-preserving and blockchain-inspired data sharing and governance in smart city environments, which has been tested through a prototype founded on a simulation model.

In this thesis work, blockchain is considered a design inspiration, rather than a fully developed infrastructure. A full decentralized blockchain system requires peer-to-peer nodes, a consensus mechanism, deployment of smart contracts, transaction cost measurement, and network-level measurement, and such aspects are not included in this developed prototype. DataTrust_SC essentially simulates selected features of blockchain with a backend policy logic, simulated smart-contract functionality, a hash of data, a hash-chained audit trail and verification of the audit trail. It is important to emphasize such a difference so that the work will not be oversold as having implemented a real blockchain infrastructure when actually only a simulation is carried out.

It is indeed the legal, ethical and regulatory constraints on the fact that the data that can be used in a smart city – namely, data about the movement of people, the usage of resources and statistics on the use of the service within a city- are very sensitive and linked to personal privacy, which limits real-world data accessibility for use within the context of a smart city. Even though the test performed here relies on simulated smart city data, it does provide a valuable foundation on which further tests and feasibility assessments can be performed. Through simulation, we are able to simulate the data as it would be found in a real smart city environment while minimizing privacy concerns and providing a higher level of reproducibility than would be found in experiments involving real smart city data. It is with the simulated data that we test the ability of our framework to handle the encrypted storage, selective access and auditability required to support our decision support systems, and our dashboards.

1.2 PROBLEM STATEMENT

Smart cities are highly dependent on efficient data sharing across a range of stakeholders such as government, service providers, private enterprises, citizens and so on. However, almost all current smart city platforms operate in a centralized data sharing environment, whereby data is collected, managed and stored by a trusted single authority or by a few select proxy parties. Such a centralized approach to data sharing, while easier to handle from an aggregation and management perspective, suffers from critical shortcomings with regard to security, privacy, scalability and trust. Such limitations are extremely detrimental to the overall reliability and trustworthiness of smart city systems where large amounts of sensitive data are involved (Allam & Dhunny, 2019).

The vulnerability to single points of failure, one of the most significant threats in the centralized data sharing model, makes centralized data repositories an ideal target for cyber-attacks such as malware, hacking and denial of service. An effective malicious attack on a centralized data repository can lead to immense data loss, breakdown in

services and an overall failure of system availability. Researchers have previously highlighted how centralized smart city platforms are particularly susceptible due to the highly valuable nature of the data and the interdependence between various urban services. This vulnerability directly impacts critical city infrastructure like transportation, energy and health services.

A data privacy leak, on the other hand, is another crucial security challenge that centralized smart cities must confront. Data that is acquired from smart cities usually comprises personal and sensitive information categorized by Personally Identifiable Information (PII), geographical location, user profiles and logs, medical records and usage records. In a centralized environment, it is highly complex to apply privacy protection and avoid unwarranted inference attacks. It is known that even an anonymized dataset can be reverse-engineered by using sophisticated data analysis techniques, which can eventually reveal private personal data. For this reason, privacy cannot rely solely on anonymization; it must also integrate encryption, access controls, audit trails and solid governance.

Another area where data privacy and security issues overlap is the concept of unauthorized access. Traditional centralized systems rely on access controls implemented by the central authority, responsible for managing the authentication and authorization of users to required data. However, software vulnerabilities, human error or intentionally malicious actions on the part of insiders may lead to the violation of security policies. Standard role-based access control systems are typically not adaptable to smart cities; fine-grained access controls must be implemented. With the growing number of users/services in a smart city system, security and transparency in access controls will be a major issue.

Another significant security challenge raised by centralized smart city systems is the manipulation and integrity attack on data. Centralized data updates are generally only assigned to limited administrators, and potentially malicious parties can alter, modify or erase data without any real-time record. Unmitigated malicious modifications of data in a smart city can have catastrophic consequences such as inaccurate transportation management instructions, false environmental reports and incorrect census information. Ensuring the data integrity throughout its entire lifecycle is a persistent challenge with conventional centralized systems (Singh et al., 2022).

In addition to the technical limitations stated above, the centralised system is bound to suffer from a trust issue between all stakeholders. Data providers might be reluctant to share sensitive data with a single authority because they are afraid that it could be used maliciously, accessed in an unauthorized manner or that they will lack insight into the usage or future processing of their data. Data consumers may distrust the data offered in the central repository. Such a trust problem negatively influences the cooperation between stakeholders, the availability of data and the efficient deployment of data-driven smart services (Lu et al., 2019; Yadav et al., 2023). This limitation is especially problematic in a complex multi-stakeholder environment where different actors possess varying levels of technical knowledge and differing interests.

In addition, systems subjected to centralization are opaque and, as a result, offer a limited verifiable audit trail and accountability. Stakeholders are also not usually aware of who has accessed certain data, when it has been accessed, and what was the

reason. This may not be very transparent, which makes it hard to comply with the requirements of the regulations and makes governance mechanisms weaker. Without tamper-proof logging and decentralized verification, the conflict over access rights to and the use of data will be hard to solve objectively (Casino et al., 2020).

These problems are also complicated by scalability. With the growth and development of smart cities, the amount, speed, and diversity of data grow exponentially. The centralized infrastructures are inefficient in scaling and ensuring security and privacy conditions. System performance and user experience can be impaired by the presence of performance bottlenecks, latency increase, and an increase in operational costs (Khan et al., 2022). They are limiting factors to the long-term viability of centralized smart city data platforms.

All these issues indicate a basic incompatibility between the needs of contemporary smart city environments and the potential of traditional centralized models of data sharing. The existence of a decentralized, secure, and privacy-sensitive data-sharing architecture with a limited dependence on a trusted intermediary, fine-grained access control, data integrity, and established audit trails is necessary. The need to address these issues is the central reason why this study was chosen, and the inception of the proposed DataTrust_SC framework conceptually applies the principles of blockchain-based logging and cryptographic techniques to address the drawbacks of centralized data sharing of smart cities and proves its functionality with a simulation prototype.

The problem addressed in this thesis is therefore not the deployment of a complete blockchain network. Instead, the problem is how to design and validate a privacy-conscious and blockchain-inspired smart city data-sharing framework that supports encrypted storage, fine-grained policy enforcement, dataset integrity reference, accountable audit logging, and simulation-based evaluation. This framing is important because the implemented prototype does not include real distributed consensus, deployed blockchain smart contracts, or real gas-fee measurement. The contribution should therefore be understood as an integrated simulation of blockchain-inspired data-sharing mechanisms rather than as a fully operational blockchain system.

1.3 RESEARCH MOTIVATION

The growing dependence on data-intensive applications of smart cities has increased issues pertaining to security, privacy, trust, and governance of social data. Although stakeholders in smart cities acknowledge the importance of collaborative data sharing, current centralized and semi-centralized solutions have not been able to adequately address these issues, thus resulting in low adoption and highly fragmented data ecosystems. The current study is driven by the fact that it is necessary to create a reliable, privacy-saving, and decentralized data-sharing system that supports the situation-specific operational and management needs of today's smart cities.

RM1: BLOCKCHAIN ADOPTION MOTIVATION.

The use of blockchain technology has become a potential conceptual framework for managing trust in a decentralized fashion because of its characteristics like

immutability, transparency, and cryptographic integrity. The concept of blockchain is applied in the first research at the architectural level to create a data-sharing structure with the minimum number of trusts, and the prototype provided in the research emulates the process of blockchain-style audit records and authorization in a controlled setting. However, this thesis does not claim that a real blockchain network is deployed. The motivation is to use blockchain-inspired principles for auditability, traceability, and deterministic policy enforcement while keeping the implementation scope realistic.

RM2: PRIVACY PRESERVING DATA SHARING MOTIVATION.

The data in smart cities often consists of sensitive personal data concerning the identity of the citizens, locations, behaviors, and patterns of use of the provided services. The uncontrolled exposure of such information may result in privacy invasion, risk to surveillance, and non-compliance with regulations. Hence, it is not just a technical but a socio-legal obligation to make sure that privacy is preserved. The necessity to design mechanisms of sharing data that would secure sensitive information and allow authorized use of data for public service improvement motivates this research.

RM3: FINE GRAINED ACCESS CONTROL MOTIVATION.

The prevailing access control systems on smart city platforms are mostly rough-grained, unchanging, and centrally imposed. These mechanisms do not have the dynamism and versatility to accommodate dynamic, context-sensitive, and multi-stakeholder access policies. The scenarios of the smart cities require fine-grained access control in order to take into consideration user roles, attributes, purposes, and time constraints. This study is inspired by the necessity to implement access-control policies in a transparent and accountable way and at the same time transparent and accountable.

RM4: MOTIVATION TO ADDRESS TRUST DEFICIENCY AMONG STAKEHOLDERS

There is a great impediment to inter-organizational data sharing due to the lack of trust between the stakeholders of smart cities. Fear of misuse, data loss, or visibility of data seems to be one of the primary reasons why data providers are afraid and would not share with others. The ability of blockchain to provide transparent and immutable logs can assist in building trust by making sure that any data access logs can be verified and cannot be repudiated. The rationale behind this study is the necessity to establish a data-sharing setting with the minimal level of trust that would promote cooperation without the necessity to blindly trust centralized authorities.

RM5: MOTIVATION TO OVERCOME LIMITATIONS OF EXISTING SOLUTIONS

Despite multiple blockchain and non-blockchain-based solutions of data-sharing being suggested, most of them are limited by high computational costs, the absence of privacy guarantees, the inability to scale, or irrelevance to real-world smart city applications. The available solutions mainly deal with security or privacy individually and not holistically. The rationale of this research is the desire to come up with an integrated system that will take into account the concerns of decentralization, privacy, access control, data integrity, and performance efficiency. At the same time, this thesis

also recognizes that the actual prototype is a simulation, not a deployed blockchain network. This means that unimplemented parts, such as real smart contracts, consensus mechanisms, and real blockchain gas analysis, are treated as limitations and future work.

1.4 RESEARCH OBJECTIVES

According to the outlined issues and driving forces, this study has the following objectives:

1. **RO1:** To design a privacy-conscious and blockchain-inspired smart city data-sharing framework that combines encrypted data storage, fine-grained role- and attribute-based access control, and accountable audit logging while clearly distinguishing the conceptual blockchain design from the implemented simulation prototype.
2. **RO2:** To implement the DataTrust_SC prototype using a web-based simulation architecture that supports AES-based dataset encryption, hash-based dataset identification, simulated PolicyContract-style authorization, grant/deny access decisions, hash-chained audit records, and audit-chain integrity verification.
3. **RO3:** To evaluate the prototype in a controlled simulated smart city environment by testing authorization correctness, unauthorized access rejection, audit log completeness, simulated authorization latency, simulated transaction-cost indicators, dataset distribution, and disaster monitoring behavior, while acknowledging the absence of real blockchain consensus, deployed smart contracts, and real smart city datasets.

All three objectives inform the design, implementation, and evaluation of the proposed DataTrust_SC framework without claiming that the prototype is a fully decentralized blockchain system.

1.5 RESEARCH QUESTIONS

In order to answer the research questions in a systematic approach, this thesis aims at answering the following research questions:

1. **RQ1:** How can a privacy-conscious and blockchain-inspired architecture improve secure and accountable data sharing among smart city stakeholders without requiring full decentralized blockchain deployment?
2. **RQ2:** How can AES-based encryption, RBAC/ABAC-style access control, simulated PolicyContract authorization, and hash-chained audit logging be integrated into a working smart city data-sharing prototype?
3. **RQ3:** What do the controlled simulation results show about the prototype's authorization behavior, unauthorized access rejection, audit completeness, simulated latency, simulated transaction-cost indicators, and disaster monitoring functionality?

These three research questions align directly with the three research objectives and keep the research scope limited to the implemented simulation prototype.

1.6 SCOPE OF THE RESEARCH

The study described herein is intended for the design, implementation, and verification of a blockchain-inspired architecture that provides smart city data protection, sharing of smart city data, and privacy to privacy-sensitive data in a smart city. Smart cities systems are multidisciplinary in nature, hence the need to define the scope of this research so that it is analytical, methodological and viable.

INCLUDED SCOPE

The following constitute the main aspects included in this study:

Firstly, this study presents the examination of the usage of blockchain as an abstract trust management model in smart cities' data sharing. It considers characteristics of blockchain, such as transparency, tamper-evident logging, and trust-minimization in the smart cities context for overcoming security and accountability challenges of multi-stakeholder urban ecosystems (Casino et al., 2020; Zhang et al., 2023). These properties of the blockchain have been modeled in the prototype using structured audit logs and hash-based data-set identifiers, rather than by having a distributed consensus network in operation.

Secondly, this paper deals with policy-based access authorization mechanism design and its implementation in a prototype with the use of smart contracts. These smart contracts are not deployed onto any blockchain network; rather, policy enforcement is simulated using backend evaluation and structured audit logs in order to examine access control processes (Xu et al., 2021).

Thirdly, the research area is surrounded by privacy-preserving data sharing for securing sensitive data using cryptographic techniques like encryption, key management and disclosure control in smart cities, such that potential threats like unauthorized data access and privacy loss are minimized while providing privacy to authorized users who can benefit from shared information in a secure way. Care is taken for citizen data and operational data confidentiality throughout the data lifecycle from generation to consumption and audit trails (Rahman et al., 2021; Lu et al., 2019).

Fourthly, analysis and test of the proposed framework are done with the simulation-based security and performance, taking measurements of simulated authorization latencies, cost of a transaction in a simulation and audit log accuracy (Khan et al., 2022). In this thesis, the results of the performance measurements are identified as simulated parameters rather than real gas costs for blockchain transactions or actual network latencies on a distributed network.

Fifthly, performance testing and validation of the proposed architecture are undertaken with the aid of simulation on synthetic data representative of smart city data. While the actual architecture would be utilized over a smart city infrastructure, the purpose of this study is to investigate the data classification, authorization mechanisms, data sharing possibilities and audit capabilities on simulation data alone.

Sixthly, this study incorporates other elements such as a simulated PolicyContract component, contract identifiers, chained data audit logs (such as previousHash, entryHash, and chainIndex) and a chain verification interface and provides for

frontend presentation of contract and chain details in a simulation-based demonstration.

EXCLUDED SCOPE

The smart city concept involves a lot of fields; some specific fields are excluded from this study to prevent its vague and extremely wide research domain.

The current study has excluded issues concerning the design of hardware at IoT devices, such as the sensor and related embedded systems, or the communication devices at the device level. Factors like the precision of sensors, power consumption of IoT devices, and security of the devices at the hardware level are not part of the present research area, as these are highly correlated with IoT hardware design and require sophisticated engineering approaches.

This study does not delve into the design of sensor-fabricating devices, their deployment mechanisms and the protocols used at the IoT device communication layer, such as the design at the physical layer, routing protocols, and optimization of wireless signals. This study assumes the existence of operational smart city data sources providing digital data streams.

The study does not research urban policy and its legal implementation or governance frameworks other than those that directly impact technical smart city data security and data privacy. Legal, financial and political regulations aspects are only discussed in an abstract sense, without going into deep analysis, as the core contribution of the paper lies in the technical aspect of design and testing of systems.

The exclusion scope explicitly indicates the exclusion of the deployment of actual blockchain networks. No peer-to-peer validator nodes, mining procedures, staking processes, digital currency transaction operations, storing data directly onto a public blockchain network or on networks such as Ethereum, Polygon or Hyperledger, and deploying smart contracts onto these networks have been simulated in this work. In turn, measurements on transaction costs and authorization latencies are reported as parameters from simulations.

The current study does not provide formal cryptographical proof, penetrative analysis of security vulnerabilities, production certification for system security or evaluation on real legal and regulatory standards, though these are essential future works. This current contribution lies on the limited simulation prototype that implements encryption, role and attribute-based access control, simulated smart contract type policies, chained hash-based identifiers and disaster monitoring for smart city data management.

1.7 CONTRIBUTIONS OF THE THESIS

This thesis includes a few original and valuable contributions to the nature of smart cities' data management and secure distributed systems. These are the main contributions that can be summarized as follows:

Privacy-Conscious and Blockchain-Inspired Framework

The first contribution is the proposed DataTrust_SC framework. This study presents DataTrust_SC, a privacy-conscious model to use in sharing data in smart cities securely and responsibly. The framework combines encryption-based confidentiality, fine-grained access control, and blockchain-inspired audit logging in one workflow. In this thesis, the contribution is not claimed as a full decentralized blockchain implementation but as a blockchain-inspired simulation framework.

Fine-Grained Access Control Mechanism

The thesis proposes a role- and attribute-based access control mechanism enforced by the backend and inspired by smart-contract logic. The method provides deterministic decisions on authorization, transparency, and auditability of a simulated environment without having to run it in a live blockchain network (Xu et al., 2021). This contributes to more precise access decisions compared with broad role-only control.

Privacy-Preserving Data Flow Mechanism

A safe data flow model is created in which encryption, regulated data disclosure, and authorization based on policy are integrated to prevent loss of privacy when sharing data. The prototype applies the model by simulating blockchain-style accountability through audit logs and hash-chained records. The framework enables data to be provided only to authorized users, and the access process can be checked through logs. The framework also uses the principle of least privilege, which is a compromise between privacy and usability (Rahman et al., 2021).

Simulated Contract-Like Policy Execution

The prototype contributes a simulated PolicyContract component. Each dataset can be associated with a policy-contract instance created from the dataset policy, owner role, and data hash. The component locks the policy object after creation, evaluates the requester's role and attribute values, returns a structured decision object, and provides a contract ID. This improves the clarity of the smart-contract-inspired part of the system while avoiding the false claim that real smart contracts were deployed.

Hash-Chained Audit Logging and Verification

Another contribution is the improvement of the audit log from a flat MongoDB collection into a hash-chained structure. Audit records can include previousHash, entryHash, chainIndex, and contractId. The chain verification function checks whether each entry is correctly linked to the previous entry. This provides tamper-evident auditability within the simulation environment, although it does not provide the same guarantee as distributed blockchain consensus.

Simulation-Based Evaluation and Disaster Monitoring

The framework is tested through controlled simulation using synthetic smart city data. The evaluation includes authorization correctness, unauthorized access rejection, audit logging behavior, simulated transaction-cost indicators, simulated authorization latency, dataset distribution, and disaster monitoring. The disaster warning and aid

center module demonstrates how governed smart city data can support threshold-based urban alerts and safety-related decision support.

Clear Implementation Boundary and Scope Definition

A final contribution of the thesis is the explicit distinction between conceptual blockchain design and the implemented prototype. The work clearly states that real consensus, deployed smart contracts, decentralized validation, real gas fees, real smart city datasets, and production-level security certification are not implemented. This improves the academic accuracy of the thesis and directly addresses the risk of overstating the contribution.

1.8 THESIS ORGANIZATION

This thesis comprises seven chapters.

CHAPTER 1: INTRODUCTION

Chapter 1 describes the background of the study, problem statement, motivation of the study, objectives of the study, questions of the study, scope of the study, contributions of the study, structure of the study, and clarification of DataTrust_SC being blockchain-inspired as a simulation prototype rather than a real decentralized blockchain.

CHAPTER 2: LITERATURE REVIEW

Chapter 2 is about the literature review. Smart city data management, data sources, smart city data privacy and security issues, basic knowledge of blockchain, smart contracts, smart contracts in data sharing, access control, privacy preservation methods, and research gaps are discussed in this chapter.

CHAPTER 3: SYSTEM MODEL AND PROBLEM FORMULATION

Chapter 3 presents the system model and problem formulation. It includes the main components of the system framework, Data Providers, Data consumers, Policy and audit layer, encrypted data storage, Threat model, Assumptions, Requirements of the system and formalization of access control.

CHAPTER 4: PROPOSED ON FRAMEWORK DATA TRUST_SC

Chapter 4 elaborates on the proposed DataTrust_SC framework. It includes the architecture of the framework, privacy-preserving mechanism, access control model, data policy enforcement workflow, representation of the simulation prototype, security mechanisms of the framework, and the boundaries between the proposed system and implementation.

CHAPTER 5: IMPLEMENTATION AND EXPERIMENTAL SETUP

Chapter 5 discusses the implementation and experiment setup. The frontend of the application is the React framework, the backend framework is Node.js and Express, and the storage used is MongoDB. The encryption algorithm employed in the system is AES, the hash algorithm used is SHA-based, the access policy matrix of the

framework is defined as well, the simulation implementation of Policy Contract, the hash-chained audit log and the simulation of the disaster warning message generation and the dataset of the system are done. Various experimental scenarios are set up, and performance evaluation indicators are defined.

CHAPTER 6: RESULTS AND PERFORMANCE EVALUATION

Chapter 6 talks about results and performance evaluation. This chapter proves that the system can authorize users correctly to access the data resources and deny unauthorized access attempts. The audit log is validated against the recorded records, and the integrity of the audit log is verified by checking the hash chain. The simulated cost of gas consumed by the smart contracts is analyzed. Simulated authorization latency is estimated. Statistics of the distribution of data resources and details of the warning messages for the disaster are presented. At the same time, simulated gas and authorization latency values presented in this work are only estimates based on the structure of smart contracts and cannot be interpreted as real data on a blockchain.

CHAPTER 7: CONCLUSION AND FUTURE WORK

Chapter 7 is the conclusion and future work. It concludes the entire thesis, lists contributions, analyzes limitations, and proposes future work on a real blockchain testnet, smart contract implementation, intensive testing on security, verification on real smart city data, large-scale evaluation of the system, framework integration, deployment and advanced privacy techniques in access control.

CHAPTER II

LITERATURE REVIEW

2.1 INTRODUCTION TO THE LITERATURE REVIEW

Recent technological advances have significantly influenced urban context in terms of shaping the smart city concept which involves the integration of information and communication technologies, data analytics, Internet of Things, intelligent systems, and networked public services in order to improve the quality of urban life. With the rise of the concept of smart cities, data has become a critical resource in improving transportation, energy management, environmental monitoring, public services, public safety, infrastructure planning, and citizens' participation. Smart city literature is inter-disciplined across urban planning, computer science, information systems, information security, public administration, and data governance, etc (Albino et al. 2015).

A smart city is based on the effective collection, processing, and sharing of vast amounts of heterogeneous data. Data can be collected from IoT sensors, public security cameras, city traffic cameras, agencies, public meters, service applications, etc. Data obtained could enable better decision making, but has issues concerning the citizen's privacy, security, accountability and transparency, and trust of stakeholders. Therefore, the literature on smart city data sharing should be investigated not only from the standpoint of technological performance, but also from the standpoint of data governance, with appropriate access control.

The literature survey is conducted in the research areas that provide a basis for the developed DataTrust_SC framework. The paper reviews smart city data management, privacy and security challenges, fundamentals of blockchain, data sharing schemes, data access controls, privacy-enhancing methods, auditability, simulation-based evaluation, and research challenges. All these are directly related to this thesis's objective of designing and evaluating a privacy-awareness and blockchain-inspired data sharing framework for smart cities.

One significant theme is the negative aspects of a central smart city system. Although centralizing system design is attractive, it can also lead to negative implications such as; data security & integrity issues, single points of failure, privacy issues, improper usage, and non-transparency & auditability. In multi-stakeholder smart city systems, these problems are even more critical due to data being passed between many individual, dispersed private & public organizations, academics, utility providers, and individual users. Without a trust factor between all stakeholders in the system with regard to data sharing the system cannot be sustained (Yadav et al., 2023).

Thus, the benefits of blockchain technology to provide a form of account and traceability in a data sharing system are being identified such as securing data with hash-based functions and storing them in append-only logs, the transparency of transactions, and smart contracts which can enforce business logic. The disadvantages that hinder using blockchain directly in this system include: overheads in storage space, limited scalability and low latency, the privacy of data, abuse of smart

contracts, and the cost of deploying such a system. Therefore it should be used as principles that can be adapted as needed for this particular system (Li et al., 2022).

The relationship between blockchain and the simulation prototype is emphasized. Blockchain was examined for the design of the DataTrust_SC system and it was utilized as the inspiration for the DataTrust_SC system and its principles. The prototype designed for this project is not representative of a totally decentralized blockchain application; a live blockchain application would involve decentralized nodes, a consensus mechanism, smart contracts, and distributed evaluation over a network. This prototype however only simulates these aspects as it uses a backend for policy-based evaluation, hash-based functions to define datasets, a simulated execution of PolicyContract, logs, a hash-chained log structure, and chained contents.

Another overarching topic in the literature is privacy preservation. Smart city data can identify individual users, their location, their activities, their use of public services, the operations of a household, or the status of some infrastructure. Privacy-preserving techniques, such as encryption, anonymity, access control, differential privacy, zero-knowledge proofs, and audit logging have been investigated in order to address the issues, such as illustrated by the references in (Ziegeldorf et al., 2020; Rahman et al., 2021; Kumar et al., 2021). No single method will suffice. Encryption can ensure confidentiality of the stored data, but cannot define who has permission to access it. Access control can limit the potential users, but it relies on correct and strong policy enforcement. Audit logging keeps a record of users' activity and its immutability is essential for ensuring accountability.

DataTrust_SC is placed at the intersection of the above subjects. It uses AES encryption to secure the data, role and attribute-based policies for its access control, hashing for identification of datasets, smart-contract-like policy evaluation through a simulation, and structured audit logging for the tracking of actions within the system. The simulation prototype uses prevHash, entryHash, chainIndex, contractId, and a chain verification mechanism to increase traceability in the blockchain-like audit log layer, although this does not implement a real distributed blockchain structure.

The chapter is structured as follows: Section 2.2 provides a background on the management of smart city data. Section 2.3 explains the privacy and security concerns in the context of smart cities. Section 2.4 introduces concepts on blockchains needed for smart city data sharing. Section 2.5 explains smart city data sharing mechanisms including centralized, decentralized, hybrid, and simulation-based approaches. Section 2.6 describes access control on smart city data. Section 2.7 discusses various techniques for privacy preservation in smart city data. Section 2.8 covers the idea of auditability and hash-chained logging and Section 2.9 explains simulation-based evaluation of such mechanisms and its limitations. Section 2.10 summarizes the identified research gaps and Section 2.11 concludes the chapter.

2.2 SMART CITY DATA MANAGEMENT

An efficient data management system is the core foundation of a smart city. A smart city must be capable of collecting, processing, storing, sharing, and analyzing a tremendous amount of data produced by or relevant to cities. These data are utilized to

inform a wide range of public service provision such as transport, environment, energy, security and safety, maintenance, and citizen services.

Smart city data management is particularly complex due to its dynamic, heterogeneous, sensitive, and dispersed nature. Urban data are generated by various sources such as sensors, public bodies, businesses, citizens, and automated processes; and vary widely in terms of structure (structured, semi-structured, or unstructured), ownership, timeliness, completeness, and sensitivity. As such, a smart city data sharing framework needs to consider not just technical storage and processing but also policy enforcement, data privacy, data integrity, and responsibility for data.

Smart city data management usually involves the full lifecycle of data. The data life cycle starts with data creation and collection; then transmitted, stored, processed, shared, analyzed, kept, and deleted. Each stage presents various risks. Data can be created without sufficient control, transmitted through non-secure channels, stored in plain format, or processed in a way to derive inference; and shared to un-authorized stakeholders. Security and privacy are critical at the full data lifecycle of smart city data.

2.2.1 DATA SOURCES IN SMART CITIES

Smart city data is collected from numerous varied sources, of which IoT sensors constitute one of the predominant ones. Such sensors are designed to measure environmental aspects like temperature, humidity, and air quality, but also city functions like traffic density, noise pollution, water and waste levels, and energy consumption levels. The merit of IoT sensors is in their ability to interconnect the physical world and the virtual platform and enable monitoring of city functions continuously over time.

Traffic systems constitute another chief source of data, through the use of cameras and road sensors; on the other hand, it could also include information retrieved from mobility applications or public transport platforms that would reflect data on congestion, usage of roads, behavior of traffic, usage of public transport, occupancy rates in car parks, and also accident situations. Traffic data may be useful in transportation planning and emergency response management. Nevertheless, it can also expose behavioral patterns if not managed properly.

Utility systems can also provide substantial data for a smart city. Smart meters and utility platforms can track water consumption, electricity consumption, and gas consumption levels, in addition to overall load on utility infrastructure, which allows for effective distribution of services. On the other hand, it can provide insight into the behavior of households. Thus, energy consumption level at particular time periods can expose whether anyone is in the house or if they are outside. In other words, it necessitates strict access controls.

Data is also offered by the public agencies. Data include administrative data and service-related data. Examples: citizen service requests, public safety reports, environmental data, urban planning data, public health data, infrastructure states. This data can be valuable for both governance and research, but can also contain

private/confidential personal or operational data. Thus it must be shared only according to a clear policy.

Citizens are also producers of data through mobile applications, online citizen portals, social media feeds, user feedback systems, complaints management systems, and participatory sensing tools. Citizen-generated data are useful for understanding current urban issues and public services. Citizen data includes user locations, individual identities, personal thoughts, and sensitive social data which can be a source of ethical and privacy challenges (Kitchin, 2014).

Finally, another data source is represented by private organizations. Transport, telecommunication, and energy utilities, logistics service providers, and digital platforms may possess urban data valuable for the city. However, they may hesitate to share them due to commercial interests, legal requirements, or absence of trust in central systems. Therefore smart city data sharing systems need to guarantee that public and private parties can share their data with transparency and control.

This heterogeneity of smart city data sources leads to the problem of interoperability and governance. Each source has different formats, data access controls, standards, or privacy expectations. Hence a safe smart city data sharing infrastructure must be able to provide means for storing metadata, policy descriptions, roles, dataset types, sensitivity level, and audit log.

2.2.2 DATA TYPES IN SMART CITIES

Smart city data can be classified in many ways. One typical classification is by structure of data, into structured data (such as numeric value, database record, smart meter readings, sensor data, etc), semi-structured data (such as log files, metadata, form data, machine-generated record, etc) and unstructured data (such as image, audio, video, text document etc). Structured data, semi-structured data, and unstructured data are involved in a smart city which makes a unified smart city data environment hard to manage (Khan et al., 2021).

Another common way to classify smart city data is by domain, such as environment data (air quality, temperature, humidity, polluters density, noise level, rainfall level, flood indicator, etc.), traffic data (road traffic congestion, traffic density, traffic accident reports, travel time, public transport / smart meter's status and mobility, smart parking device, etc.), utility data (water, electricity, gas consumption and distribution, etc.), citizen service data (citizen complaints, city requests, response from city, etc.) and public safety data (alarm and alert, accident report, monitoring etc).

Different types of data have different security and privacy risks. Environmental data is considered low-sensitivity at the bulk level, however individual environmental readings at a certain location provide explicit risk for the neighborhood area. The goal of collecting traffic data can be to reduce traffic congestion, however, it's possible to recover a citizen's trajectory from the mobility data. Smart meter/utility data helps save energy, however, data on electricity/gas consumption levels per household reveal the lifestyle of the residents at home. Citizen service data are generally regarding citizens' problems, and private issues like health conditions and social vulnerabilities may be contained in the data.

Different levels of sensitivity are under different security/privacy risks and only one level access rule can't tackle this requirement. A smart city data sharing system has to develop fine-grained access policies. For example, citizens should not be able to access their private utility consumption data but can have access to public environmental data. The data of some fields should be visible to researchers, but their rights should only be limited to the specified dataset, not for all data in the system. The access rights of the city authority are the highest but this should also be recorded. Though open data such as public and non-sensitive data would benefit transparency and innovation, uncontrolled sharing of these data can pose a security risk of data linking and re-identification (Cui et al., 2023). DataTrust_SC considered this requirement and made the category and sensitivity of the dataset part of the access control policy. The prototype takes environmental data, traffic data, utility data, and citizen service data as a category of dataset, citizen, researcher, and city authorities as roles. This gives finer-grained control than is solely based on role.

2.2.3 SMART CITY DATA LIFECYCLE

The data lifecycle in a smart city refers to the path followed by the data as it moves from its creation to its ultimate storage, disposal, or usage. It plays an important role in the context of a smart city with regard to the quality, security, privacy, and accountability of the data. The data lifecycle in a smart city consists of data creation, data collection, data transmission, data storage, data processing, data sharing, data analysis, data retention, and data disposal.

The initial stage of the smart city data lifecycle involves the creation and collection of data. This initial stage refers to the time where data or raw information is generated by sensors, devices, applications, public systems, users etc. The risks that can arise at this stage involve inaccurate data being collected, excessive data being collected, unauthorized devices connecting (which means the data is improperly collected) etc., thus affecting the subsequent decision-making process. It is vital that the quality of the data is maintained at the very initial stage in order to appropriately make decisions at a later stage (Zhou et al., 2022).

The next stage is data transmission. This refers to the stage where the data is transmitted from devices or platforms to the cloud systems, databases, servers, or edge nodes. At this stage, interception, manipulation, and replay attacks are a major risk, and thus data should be transported over secure communication channels in order to avoid information leakages or manipulation. However, the DataTrust_SC prototype focuses on the storage, access control, and audit logging parts and not on the lower-level issues concerning network security.

The second essential step is storage. Data in smart cities may be stored on on-premise databases, the cloud, distributed systems, or a mixture of all three. If data is stored in plain text, compromise of a database may lead to an attacker's acquisition of huge volumes of very sensitive data; therefore encryption is vital. DataTrust_SC encrypts the payload in a dataset before storing it in MongoDB.

Processing and analysis; Raw data are converted into meaningful information. These steps include data cleaning, aggregation, filtering, visualization, analytics, and machine learning. However, these steps can result in the exposure of novel privacy

risks like tracking the activity of citizens based on combined citizen, traffic, and utility data. Access control and privacy-aware processing are therefore key.

Information sharing: This is a major step in smart city systems where data is shared with different stakeholders for purposes such as service provisioning, research, planning, and decision-making. It is in this step that unauthorized disclosure or misuse is likely to happen most frequently, especially if the smart city system involves multiple data custodians and distributed storage. It is in this step that the proposed system deals the most. The proposed system makes the access decisions by checking whether the requester's role and attributes meet the DataTrust_SC policies and makes an entry to the audit log of all the decisions that it made.

The final step involves data retention, archiving, or deletion. Proper data retention, archiving, or deletion requires data not to be retained permanently. Depending on data protection laws and regulations some data require mandatory secure deletion or limited data retention. A full-fledged legal data retention module is not provided in the DataTrust_SC prototype.

2.3 PRIVACY CHALLENGES IN SMART CITIES

Widespread use of data-driven technologies in smart cities, however, has raised significant challenges relating to security and privacy. Smart cities make use of data collected, stored, processed, and exchanged from many diverse sources such as IoT devices, government, privately owned platforms, and the public, allowing for smart city services to exist. The use of these data risks exposing individuals, organizations and the infrastructure at various levels.

Smart cities differ from ordinary information systems by being heavily based on physical space and human everyday life, thus a breach of private data within a smart city is more than just the revelation of personal routines and usage patterns, but may expose the consumption of electricity in a home, or what public services are experiencing the highest stress. An attack on security could, however, have significant impacts not only on individuals but on public safety, public transportation networks, energy systems, emergency response and so much more. These challenges should thus not be merely confined to technical spheres but must also span social and governance issues.

Key drivers in relation to the problem of privacy in smart cities are the sensitivity of urban data, the heterogeneous origin, and the multi-stakeholder nature of the data. Personal data could be acquired both implicitly and over time which poses significant challenges regarding maintaining individual control over data as it is often passive. Privacy concerns can be exacerbated due to data aggregation because sensitive data is often aggregated in central storage.

2.3.1 DATA PRIVACY THREATS

One of the major privacy risks is identity leakage in a smart city context. Identity leakage happens when an individual may be uniquely, or quasi-uniquely, identifiable from data either directly, or indirectly. Direct identifiers are items of data that may identify an individual, without combining with any other pieces of data. The examples

of direct identifiers include names, identification numbers, addresses, device IDs, biometrics, or credentials. Indirect identifiers are traces related to movement/location, habitual actions by a person, time stamp, and usage patterns. Re-identification can be achieved by joining together datasets even after the removal of all direct identifiers. Several studies have shown that anonymised mobility or service-usage data are still vulnerable to re-identification attacks through available side information. (Rocher et al., 2019; Ghinita, 2020)

Another critical privacy risk is location tracking. Many smart city services or devices collect location-based data. These systems may include a GPS receiver, a mobile phone, public transport systems, traffic cameras, smart cards, and other service platforms. An individual's continuous location trace reveals personal private information such as home address, work address, traveling habits, visited locations (doctor, temple, etc), social networks, and even political inclination. Location is an important component so data are usually sensitive, and it has already been demonstrated that continuous, fine-grained location information can lead to considerable individual privacy and civil liberty harm.

Another critical privacy risk is behavioral inference. Smart city systems may produce data without containing obvious personal identifiers yet reveal user behavior, for example, the usage of electricity and gas could indicate house occupancy, traveling patterns could reveal one's daily itinerary and service usage logs could signal the economic hardship of citizens, and environmental data combined with the user's location trace could reveal that one is staying in a high-risk district. The ML and analytical techniques applied to these data could then infer the health status, political position, economic background, and lifestyle. (Cui et al., 2022; Li et al., 2021). It proves that we need to do more than anonymisation.

Data linkage is also a significant privacy risk. Combining data sources could provide new sensitive information, that the individual dataset did not contain itself. For example, a combination of traffic data, service requests, and utility usage logs could provide one with an in-depth profile of an individual's behavior. Systems for sharing data need to prevent inference and not direct disclosure only.

Another significant privacy risk is excessive disclosure. Where policies are vague or overbroad, users may receive more data than required. For example, the need for researchers in the domain of smart cities might be only statistics (not raw data). A citizen requesting environmental data does not require private utility usage logs. This requires fine-grained access controls to be employed with the principles of minimal disclosure.

2.3.2 SECURITY ATTACKS

Unauthorized Access is perhaps the most common security threat to a smart city system, this occurs when a user or attacker can access the system and its information without the preconditions established by a policy. Examples of such threats are unauthenticated access points, compromised APIs, lack of authenticated controls, unsecured databases, system bugs, stolen accounts, or badly written policies. The sharing of data and the connection of devices to users often result in authority issues.

Data Tampering is considered the most severe security threat and attack to a smart city. Unauthorized manipulation or removal of data from the smart city infrastructure, the security risk can result in poor decisions being made; for instance, false data about traffic can impact the public if the city is congested, or falsified environmental data readings can show stable conditions where a danger exists leading to poor planning of utilities or the incorrect reporting of utility resources. Maintaining the integrity of data in a distributed infrastructure system has been documented as being difficult (Zhang et al., 2020).

To smart cities, insider threats have also been a key area of concern. This occurs when individuals who already have authorized access to a system (i.e. Operators, administrators, contractors) may deliberately misuse it to gain access to private data, or by mistake, delete data. Insider threats are difficult to detect as the action appears to be that of a normal system function and cannot be distinguished by a normal security audit system.

Availability of the smart city systems can also be undermined by Denials of service or service failures. The inability to access data sharing services could also have significant repercussions for public infrastructure and emergency services, this would need serious consideration during the design phase though this attack vector was not assessed in the DataTrust_SC prototype tests.

Weak auditability is more a governance concern: users cannot really trust what is happening in a system if there are ways of editing/deleting log files without leaving any traceable evidence of such activity. A flatter, immutable log file could be the solution.

Table 2.1: Privacy and Security Challenges in Smart Cities

Category	Threat / Attack	Description	Potential Impact
Data Privacy	Identity leakage	Re-identification of individuals from direct or linked data	Loss of anonymity and identity exposure
Data Privacy	Location tracking	Continuous monitoring of movement and location	Surveillance and profiling
Data Privacy	Behavioral inference	Inferring sensitive information from patterns	Privacy erosion and discrimination
Security	Unauthorized access	Access without satisfying policy requirements	Data breach and misuse
Security	Data tampering	Unauthorized modification or deletion of data	Incorrect decisions and system failure
Security	Insider threats	Misuse by authorized users	Large-scale compromise
Governance	Weak auditability	Missing or modifiable access records	Reduced accountability
Governance	Centralized trust	Dependence on one authority	Reduced stakeholder confidence

In this section, the privacy and security issues indicate that smart city data sharing is susceptible to attacks based on security itself and also to privacy attacks stemming from the analysis and inference of the data. Standard centralized security measures cannot fully solve the issues mentioned above, because they usually lack transparency, resistance to tampering, and minimality of trust. DataTrust_SC proposes solutions in the forms of encryption, role- and attribute-based enforcement of policies, hashing of datasets, and blockchain-based logging mechanisms.

2.4 BLOCKCHAIN FUNDAMENTALS

Blockchain has emerged as a method for maintaining immutable, verifiable, and accountable data. At a high level, a blockchain is a list of records that have been connected using cryptographic hash functions. On a fully decentralized blockchain, the ledger is hosted and verified on multiple nodes according to a consensus mechanism which allows anyone to inspect transactions without depending on a central administrator.

The relation between blockchain technology and smart city data sharing, arises from the principles of blockchain that can help to achieve trust and accountability in this context. Such principles can be seen as append-only records, hash-based integrity, traceable transactions, deterministic policy enforcement, and smart contracts. Although blockchain technology can achieve trust minimization and accountability in multi-stakeholder contexts, practical concerns about its cost, latency, and scalability limitations arise (Khan et al., 2022).

For this thesis, blockchain is considered a source of ideas for the development of the DataTrust_SC framework, and not as a fully implemented infrastructure. In fact, the developed prototype includes only a subset of blockchain-inspired features and does not implement consensus mechanisms, distributed nodes, or deployed smart contracts.

2.4.1 BLOCKCHAIN ARCHITECTURE

Data is saved on the blocks on the blockchain and chained one to another, as a new block consists of the previous block's hash. Indeed if the previous data was modified, its hash value will be modified, and this is transmitted to all other blocks thus creating a data chain which cannot be altered, thus creating a similar historic data keeping as an immutable one (Zheng et al., 2017).

Hashing is one of the core features that a blockchain needs to incorporate. A hash process takes input information and outputs a fixed-length hash value. The hash process is one way and it is based on a cryptographic hash function. The hash value of an input cannot be determined from the output hash value, however, any modification to the input data will alter the hash value of the message and be detectable. Thus hashing can be used in the context of data-sharing to ensure the integrity of the data, to detect if any unauthorized modifications to the data have occurred, and also to link records in the audit trail. It is a very useful feature in protecting historic data and detecting tampered data within smart city systems (Kumar et al. 2023). Hashing is used in DataTrust_SC to establish links between the different data sets and to form hash-chained audit entries.

Another essential feature in a full blockchain is the consensus, which aims at letting all the nodes agree on a consistent state of the ledger (e.g. Proof of Work, Proof of Stake, Practical Byzantine Fault Tolerance). It eliminates the requirement of the existence of a third trusted party but may come with a high price in terms of costs (computations), communication overhead, and latency (Li et al. 2021). No consensus algorithm is integrated into DataTrust_SC thus this system is not a full blockchain.

The use of on-chain or off-chain data storage is also an important aspect. As smart city systems generate massive data, it is not feasible to store all of the data on-chain due to storage capacity limits, cost, and privacy concerns. In many scenarios, hybrid systems, which link off-chain massive data through the data on-chain (i.e. metadata contains the hash of the data set, etc.), are adopted and seem more appropriate for smart cities systems dealing with large data sets like sensor data, images, etc. (Kumar et al. 2021; Li et al. 2022). This feature is taken into consideration at a high level in DataTrust_SC, as the data sets are encrypted and stored in MongoDB, and the blockchain is simulated by the use of hashing and chaining of audit entries.

2.4.2 SMART CONTRACTS

Smart contracts are simply contracts to be executed when a certain condition is met. In an application of blockchain, it can perform different actions, such as authentication, authorization of data sharing, transaction, and logging of the audit trail. Especially for the usage of data sharing in smart city applications, it can verify if the access requirement of the requesters has been satisfied before returning the data.

Smart contracts provide an advantage of determinism enforced by the rules. The output is supposed to be the same if we provide the same inputs to the same policy. Especially useful for access control and avoiding inconsistent judgments. And it has the advantage of accountability; by keeping a log of commands that have been executed. Many smart contract-based access control methods were proposed such that automatic authorization and fine-grained policy enforcement can be achieved.

However, a "true" smart contract has to be executed in a blockchain environment and it needs to be deployed, tested, secured, incur transaction cost, and have its metadata revealed, and also there could be a bug in it. Thus it's not suitable to implement the system using a smart contract during the initial prototype due to the problems mentioned. The crucial strengths of smart contracts are: irreversibility for audit trail, conditional data sharing, and interoperability, which are all only feasible in true smart contract execution. (Khan et al., 2022; Zhang et al., 2023).

The smart contract in DataTrust_SC is realized as a PolicyContract backend class which acts as a smart contract's functional imitation. A certain policy is generated for each data set, and after that, the policy is fixed. The policy is then checked against the role and attributes value of the requester, and then the corresponding answer along with the contractId will be returned, it is not the real smart contract.

Table 2.2: Blockchain Components and Smart Contract Functions

Component	Description	Relevance to Smart Cities	DataTrust_SC Interpretation
Block	Stores transaction or event data	Can represent access events or metadata	Simulated through audit entries
Hashing	Cryptographic digest of data	Supports integrity and tamper detection	Used for dataset and audit-entry hashes
Consensus	Agreement among distributed nodes	Reduces reliance on one authority	Not implemented
On-chain storage	Data stored directly on blockchain	Useful for small metadata or logs	Conceptual only
Off-chain storage	External storage for large data	Practical for smart city datasets	Implemented using encrypted MongoDB storage
Smart contracts	Self-executing policy logic	Automates access decisions	Simulated through PolicyContract backend logic
Access rules	Conditional authorization logic	Enables fine-grained sharing	Implemented using RBAC/ABAC-style checks
Audit trails	Record of interactions	Supports accountability	Implemented through structured and hash-chained logs

From this table, it is clear what the scope of work is. DataTrust_SC implements some aspects of blockchain-like features, but it does not implement all components of a true blockchain system.

2.5 BLOCKCHAIN FOR DATA SHARING

Blockchain has emerged as an attractive field for secure, responsible data sharing, particularly among entities with a lack of overall trust (multiple stakeholder domains). In the smart city context, these stakeholders include governmental institutions (both public and private sectors), universities, utilities, and citizens. Blockchain, or a blockchain-inspired mechanism, is well suited to allow logging of access, validation of data integrity, and removal of the reliance upon blind trust in a central authority.

Despite this potential, it is important that 'blockchain for data sharing' is properly contextualized. A full implementation of a blockchain will involve infrastructure, consensus, smart contracts, node management, transaction throughput, and security testing. At present, most systems implement blockchain concepts in simulation rather than utilizing true distributed infrastructure, which is the position of DataTrust_SC.

2.5.1 CENTRALIZED VS DECENTRALIZED SHARING

Sharing of data in the traditional system is centralized to one trusted authority who collects, stores, and manages the access controls and access logs. Even though it is easy to set up and maintain, it has a few major drawbacks:

First, it is a single point of failure. A user would not be able to access any critical smart city data in the event that the central server or database is compromised. The central database might also be storing sensitive smart city data which can be exposed to attackers. Examples of fraudulent actions performed at central server premises are unauthorized access and insider fraud.

Second, it lacks transparency. A user of the system has no means of knowing if valid data access policies are being followed and if data access requests made to the data are valid. A data owner has no visibility into the number of times a particular piece of data was accessed. This lack of transparency is likely to breed distrust between the user and the smart city data provider.

Third, it raises privacy issues. Placing all the private information into a single, central point increases its visibility to an attack. A central data repository raises increased concern for easier insider fraud. Privacy issues for centrally shared smart city data are highlighted as a concern (Rahman et al., 2021).

By decentralizing the system to use blockchain, the issue of authority can be resolved. Not only does such a system allow for integrity and transparency of access history, but it also automatically enforces the data access policies through smart contracts and enables features like immutability, transparency, and trustless access controls (Li et al., 2021; Khan et al., 2022; Li et al., 2022). There are limitations on blockchain as well. The throughput of consensus is extremely slow, there is no viable scenario where a substantial amount of smart city data can be put onto the blockchain since storage capacity is limited. The metadata can be leaked via public blockchains. Vulnerabilities and loopholes exist in smart contracts that require extensive validation. Permissioned blockchains also require permissioned authorities on the nodes being utilized to participate.

Therefore, a hybrid system appears more appropriate for the two solutions described. In a hybrid system, most data stays off-chain, while sensitive and key pieces of data like data hash, metadata, data access policies definitions, and/or data access logs stay on-chain or in similar blockchain-inspired technology. DataTrust_SC is the implementation of such a system in a web-based simulated prototype.

2.5.2 EXISTING BLOCKCHAIN-ORIENTED APPROACHES

This section attempts to compare general types of approaches to data sharing that are applicable to the thesis, rather than incorporating examples of unsupportable or unsubstantiated frameworks. The comparisons that need to be drawn still exist within the literature review.

Table 2.3: Conceptual Comparison of Data-Sharing Approaches

Approach Type	Main Focus	Strength	Limitation	Relevance to DataTrust_SC
Centralized data-sharing model	One authority controls storage and access	Easy to deploy and manage	Single point of failure and weak stakeholder trust	DataTrust_SC responds by adding policy enforcement and audit traceability
Fully decentralized blockchain model	Distributed ledger with consensus	Stronger decentralization and tamper resistance	High cost, latency, and deployment complexity	Treated as future work, not implemented
Hybrid on-chain/off-chain model	Data stored off-chain, hashes or logs controlled separately	More practical for large datasets	Requires integration between storage and control layer	Used as the conceptual design pattern
Privacy-preserving access-control model	Encryption and policy-based authorization	Protects confidentiality and limits access	May lack strong audit logging	DataTrust_SC combines privacy protection with auditability
Simulation-based blockchain-inspired prototype	Tests blockchain concepts without real deployment	Useful for early feasibility validation	Cannot prove real blockchain performance	This is the implemented scope of DataTrust_SC

The comparison indicates that all three solutions have both pros and cons. Centralized solutions have good practicability while needing very high trust on a single source. Blockchain based solutions fully distributed in their most ideal forms require high implementation costs and are hard to manage. Hybrid models are the most pragmatic for smart city data, as it's unreasonable to store large datasets on-chain. Simulation based models, as seen, are useful in controlled environments. These models are, nonetheless, simulations and shouldn't be referred to as real blockchain implementations (Kumar et al., 2021).

DataTrust_SC is presented as a simulation prototype with inspiration from the hybrid approach. The project has not developed a novel blockchain protocol. Instead, it presents a workflow integrating AES encryption, role and attribute based access control and simulated contract based policy enforcement, dataset hashing, hash chaining as audit trail and controlled testing for smart city purposes.

2.5.3 DATA TRUST_SC POSITIONING AND SIMULATION BOUNDARY

The DataTrust_SC framework is best understood as a blockchain-inspired data-sharing model implemented through a simulation prototype. At the conceptual

level, it uses blockchain principles such as trust minimization, deterministic policy evaluation, auditability, hash-based integrity, and tamper-evident logging. At the implementation level, it uses a React frontend, Node.js/Express backend, MongoDB storage, AES encryption, SHA-based hashing, simulated PolicyContract logic, and hash-chained audit records.

The implemented prototype includes a PolicyContract class to simulate smart-contract behavior, and the implemented prototype also includes a chainVerifier utility to convert the flat MongoDB audit log into a hash-chained structure. Audit records include fields such as previousHash, entryHash, chainIndex, and contractId, and a verification endpoint checks whether the audit chain is intact.

The system does not rely only on a simple database audit log. It models blockchain-style traceability more clearly. However, it is still a centralized simulation because the audit chain is stored and verified in the backend environment rather than validated by independent blockchain nodes.

2.6 ACCESS CONTROL IN SMART CITY DATA SHARING

The access control defines which one may access which data and under which conditions. Data sharing in a smart city require such functions as different stakeholders have different rights and responsibilities. A citizen should not have access to all data sets, similar to a researcher, a public utility, a city authority or an operator.

RBAC (Role-Based Access Control) assigns rights based on the role of a user. RBAC is easily managed, for instance, citizens only access the environmental public data set and city authorities may access more operational data sets. For a smart city, RBAC is still too coarse-grained, access would not just depend on the role of the user.

ABAC (Attribute-Based Access Control) judges access based on the attributes of the user, resource, action, or context. Attributes associated with the user include roles, organizations, departments, or clearance. Attributes of a resource include class, privacy concerns, and so on. Context attributes include time, reasons, emergency state, and location. ABAC is more flexible than RBAC but possibly harder to manage. For a smart city, granularity is likely important, since the access permission depends not only on the data set's class and sensitivity, but on the stakeholders (Rahman et al., 2021).

The hybrid RBAC/ABAC approach combines the ease of roles with fine-grained control of attributes, appropriate for the DataTrust_SC scenario. In the prototype, the roles used by requesters are citizen, researcher, and city authority; and the data categories used by datasets are environmental, traffic, utility, and citizen service. Access decisions also take into account the sensitivity of the data, which could be public or private. This allows the system to allow public data to be accessed by a large number of requesters and private data by a few.

The implementation chapter includes an access policy matrix that models how different requester roles interact with dataset categories and sensitivities. For example, citizens can only access public records of environmental and traffic data, while private

records of utility and citizen service data are denied. Researchers and city authorities have additional constraints on how they can access data sets.

The prototype links the access control decision with the simulated smart-contract behavior. When a user makes an access request the backend can instantiate a PolicyContract based on the policy of the data set, and make the evaluation based on the role of the requester, as well as attributes. The outcome of the contract-like behavior is recorded in the audit log together with a contractId. This can also be beneficial because it enables tracking the decision to a specific instance of the policy contract that was used to create it.

It is important to note that the access control contribution is still an approximation. No operation is carried out on a blockchain virtual machine, nor is there any on-chain transaction. The access control contribution has therefore been labeled deterministic backend policy evaluation influenced by smart contracts.

2.7 PRIVACY PRESERVING TECHNIQUES

Privacy-preserving mechanisms are necessary to prevent unauthorized access to smart city data and enable its effective sharing. Various techniques are described in the literature, such as encryption, hashing, anonymization and pseudonymization, differential privacy, zero knowledge proofs, homomorphic encryption, secure multi-party computation, and access control.

Encryption is considered one of the most practical and relevant techniques. It aims at converting plain-text data into an unreadable cipher-text for unauthorized entities. Symmetric encryption is generally preferred for storing large data sets due to its performance. In DataTrust_SC, the payload of the datasets is encrypted before being stored with an AES-based mechanism and only decrypted after successful authorization.

Hashing supports integrity reference. Hashing is a mechanism by which a fixed-length digest is produced. As soon as the input text is modified, the digest will change. In DataTrust_SC, hashes are used to represent datasets and to link the audit trail. They are also used for the audit entry itself to create a chained logs structure as shown in the prototype, as well as mentioned in the literature within the context of blockchain-inspired integrity mechanisms.

Anonymization or pseudonymization is also a well-known privacy mechanism. While anonymization attempts to remove identifying attributes from data and pseudonymization replaces identifiers with artificial ones, they are considered not sufficient enough for smart cities. Indeed the spatio-temporal nature and the behavior patterns might allow for re-identifying individual citizens by combining datasets. Thus DataTrust_SC relies on encryption and controlled access (Rocher et al., 2019).

Differential privacy is applied to add controlled randomness to aggregate statistics to maintain privacy. This is good when publishing public stats but results in less accuracy and the appropriate parameterization needs to be selected very carefully. The DataTrust_SC prototype does not support this method.

Homomorphic encryption can compute over encrypted data without decrypting the data itself. This provides a strong privacy guarantee but is very computationally expensive. Secure multi-party computation allows multiple parties to compute jointly without revealing private inputs to each other. Both of these techniques are strong possibilities but neither is within the scope of the prototype implemented.

Zero-knowledge proofs allow a user to prove something to another user without revealing underlying knowledge. In a data-sharing environment, they can allow privacy-preserving authorization. These however can add complexity.

The privacy mechanism used in DataTrust_SC is a simple and bound technique: AES encryption for data, policy-based access control to determine who can access the data, and a hash-based reference rather than revealing the data's name. Audit logs allow checking what has been accessed and how often. These techniques provide a reasonable privacy assurance within the current simulation prototype but there is no formal privacy guarantee. Stronger techniques can be integrated in the future.

2.8 AUDITABILITY AND HASH-CHAINED LOGGING

Auditability is the system capability of logging and tracking system actions and events. When using smart city data sharing, a smart city system should store the following in its audit log: registration of a dataset, access request for a dataset, the role and attributes of the requestor, hash of the dataset requested, the time of access request, policy in use, and a grant/deny status. Audit logs are essential to guarantee accountability by providing a way to review how data was used.

A basic audit log uses a database to record audit entries. The basic audit log could allow an attacker to remove, alter, or reorder entries in the log. Blockchain-based systems solve this problem by having hash-linked records, each dependent on the hash of the previous one, thereby enabling traceability. This guarantees accountability and transparency of the audit logs (Casino et al., 2020; Khan et al., 2022).

The DataTrust_SC prototype implements this idea in simulation. Each entry on the audit log could contain: previousHash, entryHash, chainIndex, contractID. The previousHash would contain the hash of the previous audit log record, the entryHash represents the current log entry, chainIndex is the current position in the audit chain, and contractID will contain the ID of the simulated policy contract that generated the entry.

The mechanism to check if the audit chain has been tampered with could check if each recorded previousHash matches the calculated hash of the actual record and sort the logs by chainIndex before performing the calculation of the hashes. If the audit log was tampered with the link could be broken and thus be flagged to the system that there is a discrepancy.

This makes the prototype more resemble a blockchain-inspired system. However, this cannot be exaggerated as the audit log is still stored on MongoDB and the verification is being conducted by the backend and not independent validator nodes. The audit log is tamper-evident and not immutable.

2.9 SIMULATION-BASED EVALUATION AND ITS LIMITS

Simulation-based evaluation is a viable technique when it's not feasible to deploy the system into the real-world. Through simulation, a researcher can demonstrate the viability of the designed workflow. In DataTrust_SC, we use simulation to evaluate the procedures for dataset registration, encryption, policy evaluation, access control, auditing, analysis, and disaster monitoring.

In the implementation chapter, we describe the use of React frontend, Node.js/Express backend, MongoDB database, AES encryption, SHA-based hashing, dataset registration, access control, mock blockchain for auditing, and disaster notification module. Such information implies that the system is a working web-based prototype, rather than an imaginary proposal.

In the evaluation chapter, we check the functionality of authorization success, authorization failure, simulated gas price, authorization latency, distribution of datasets, and disaster monitoring behavior. The evaluation results prove that an authorized request can get authorized, and an unauthorized request can get denied. Moreover, it can log access events into the audit log and report simulated gas price and latency through the dashboard.

Despite the above, it should be explicitly pointed out that simulation is not reality. The simulated gas price is not real gas price for blockchain, authorization latency in the prototype cannot reflect that of in a real distributed blockchain network, a PolicyContract class written in backend doesn't mean that a deployed smart contract, MongoDB audit chain doesn't mean that a real distributed ledger, and the synthetic data used is not actual smart city data that have been generated during real operation.

As such, the simulation results may be used as proof of concept to verify that the proposed workflow for DataTrust_SC runs fine under controlled situations. The real-world blockchain performance, scalability to the whole smart city system, and security in production need further investigation.

2.10 RESEARCH GAPS

The following are the existing gaps found in smart city data-sharing research:

First, most smart city data-sharing models are based on a centralized architecture. Centralized models are easy to deploy, however, they have single points of failure and require a stakeholder to put their trust in one authoritative party. The use of a central authoritative platform will only compromise transparency and collaboration.

Second, many approaches provide privacy preservation through encryption or anonymization, yet they lack strong accountability. While confidentiality is vital, stakeholders must be aware of who has access to what data, and whether the access was legitimate. Therefore auditability must be an accompanying feature to privacy protection. (Casino et al., 2020).

Third, many systems lack fine-grained access control. Basic Role-Based Access Control (RBAC) may not be adequate, since data access in smart cities can be on the basis of the data category, sensitivity, owner of the data, requester, and purpose.

Therefore, robust RBAC/ABAC type access control needs to be implemented. (Rahman et al., 2021).

Fourth, while most blockchain-based studies describe the concept of decentralization and smart contracts, there is often ambiguity separating concept from actual implementation, which can sometimes lead to over-claiming results. This gap has been addressed, as the presented framework is presented as a simulation based and blockchain inspired model.

Fifth, many prototypes use flat records for audit trails; while flat audit trails are sufficient, they cannot strongly prove against manipulation and reordering of events. Hash-chained audit logs can provide tamper-evidence to a prototype simulation.

Sixth, testing has mainly been limited to simulation rather than to a real-world testbed due to a lack of access to smart city data. Like others, our test utilizes synthetic data. The interpretation of our results should therefore be considered from a perspective of controlled simulation.

Seventh, no actual blockchain deployment exists; this work simulates the smart contract part of the blockchain only and lacks real consensus, peers, or actual transaction fees.

Table 2.4: Research Gaps and DataTrust_SC Response

Research Gap	DataTrust_SC Response	Remaining Limitation
Centralized trust problem	Uses deterministic policy logic and audit traceability	No real decentralized consensus
Privacy risk in data storage	Encrypts dataset payloads before storage	Key management remains prototype-level
Coarse access control	Uses RBAC/ABAC-style policy evaluation	Policies are manually defined
Weak audit tamper evidence	Adds hash-chained audit records and verification	Still stored in MongoDB
Smart contract overclaiming	Describes PolicyContract as simulated	No deployed smart contract
Questionable gas-cost claims	Treats gas as simulated transaction-cost indicator	No real blockchain transaction fees
Limited dataset validity	Uses synthetic smart city data for controlled testing	No real smart city pilot
Limited security rigor	Uses mechanism-level security discussion	No penetration testing or formal proof

The research gap analysis proves that this thesis finds the correct spot. The thesis, DataTrust_SC, does not propose to solve all the data sharing problems of smart cities. Instead, DataTrust_SC is a bounded prototype with implemented encryption, policy-based access control, mock contract-based authorization, hash-based IDs, and hash-chained audit logging.

The novelty of this work is thus not to define the implementation of AES, ABAC, Hashing, or blockchain. These already exist. It consists of demonstrating the aforementioned functionalities integrated in smart cities' data sharing processes using a working simulation prototype. Hence, the contribution should be modest and accurate.

This chapter presented a literature review about smart city data management, challenges in privacy and security, blockchain basics, blockchain-inspired data sharing, access control, privacy-preserving schemes, auditability and simulation-based evaluation, and the related research gaps. It illustrated the demand for smart cities to share data in order to ensure efficient smart services, and this data sharing brings critical issues related to privacy, security, trust, and governance.

It also demonstrated the relevance of blockchain concepts due to the supporting traits such as traceability, deterministic policy execution, hash-based integrity, and tamper-proof auditability. However, the total blockchain solution is rather complicated as it needs the participation of decentralized nodes, a consensus mechanism, smart

contract implementation, actual transaction testing, and network evaluation. Hence, the DataTrust_SC proposes a blockchain-inspired simulation prototype, instead of a full blockchain implementation.

This literature review did not include non-proven named frameworks nor any claims on unverifiable examples but it showed the main concepts, mechanisms, and implementation that can be supported by the given report materials. The main research gap is that it needs to form a well-bounded integrated framework including encrypted data storage, fine-grained policy control, simulated contract-like authorization, data set hashing, hash-chained audit log, and simulated evaluation for smart cities.

Based on the above literature review, in the following chapter, the system model and problem formulation are discussed by describing the architecture of the system, threat model, assumptions and requirements, and formal access control problems for the proposed system.

CHAPTER III

SYSTEM MODEL AND PROBLEM FORMULATION

This chapter presents the system model and problem formulation of the proposed DataTrust_SC framework. The framework is designed to support privacy-conscious and accountable data sharing in smart city environments. It follows blockchain-inspired principles such as deterministic policy evaluation, hash-based integrity reference, traceable audit records, and trust minimization.

However, the implemented system is not a complete decentralized blockchain deployment. It does not implement peer-to-peer blockchain nodes, distributed consensus, cryptocurrency transactions, or deployed smart contracts. Instead, The prototype validates the proposed framework through a web-based simulation model. In this simulation, blockchain-style behavior is represented by backend policy evaluation, AES-encrypted storage, simulated PolicyContract logic, hash-based dataset identifiers, hash-chained audit records, and audit-chain verification.

The chapter describes the system architecture, threat model, design requirements, assumptions, and formal problem formulation that guide the later framework, implementation, and evaluation chapters.

3.1 SYSTEM ARCHITECTURE

The DataTrust_SC is a system for sharing smart city data among multiple authorized stakeholders such as city administrations, businesses, IoT service providers, public organizations, researchers and other authorized consumers. It is composed of four main entities: Data providers, data consumers, policy and audit layer, and encrypted data storage.

Data providers are the parties that produce and collect urban data. In this prototype, we simulate this component using devices such as IoT sensors, public administration, traffic cameras and utility meters. Before its storage, the payload of a dataset is encrypted using a symmetric AES-based encryption algorithm. Its corresponding policy and metadata also need to be stored. The metadata are attributes that describe the data, such as category, owner type (business/public/research/etc), policy specification, sensitivity level and hash reference of the data payload.

Data consumers are authorized stakeholders who request access to specific data based on their own role and attributes. A web-based interface allows consumers to access their requests. Once they initiate their requests, the backend receives it and it gets handled by the policy evaluation module. Access is granted if the request satisfies the policy related to the data, and then the data payload is decrypted. If it does not, access is denied, and plaintext is not provided.

The policy and audit layer is the core control component. It resembles principles of blockchains, such as deterministic access, traceability and tamper-evident logs. In our implementation, this component is simulated in the backend instead of a real blockchain network. The prototype enhances this layer by introducing a simulated

PolicyContract. The policy and metadata related to the data are encapsulated into a contract-like policy instance, built using the policy, the data hash and the owner role. This component takes a request that is based on the role of the consumer and the attributes they possess and returns the appropriate authorization decision.

The audit function logs critical events such as when data is registered, requested for access, granted access or denied access. Unlike flat logs in a database, each audit entry in this prototype is hash-chained together. Each audit entry can contain the hash reference to the previous entry (prepreviousHash), its own hash reference (entryHash), index within the entire chain (chainIndex) and the ID of the simulated policy contract (contractId).

The encrypted data storage component takes care of storing the encrypted payload and metadata in some type of storage. In our prototype, we simulated storage using a MongoDB database. We are storing the encrypted payloads and the related metadata, and the decryption is performed only upon authorization.

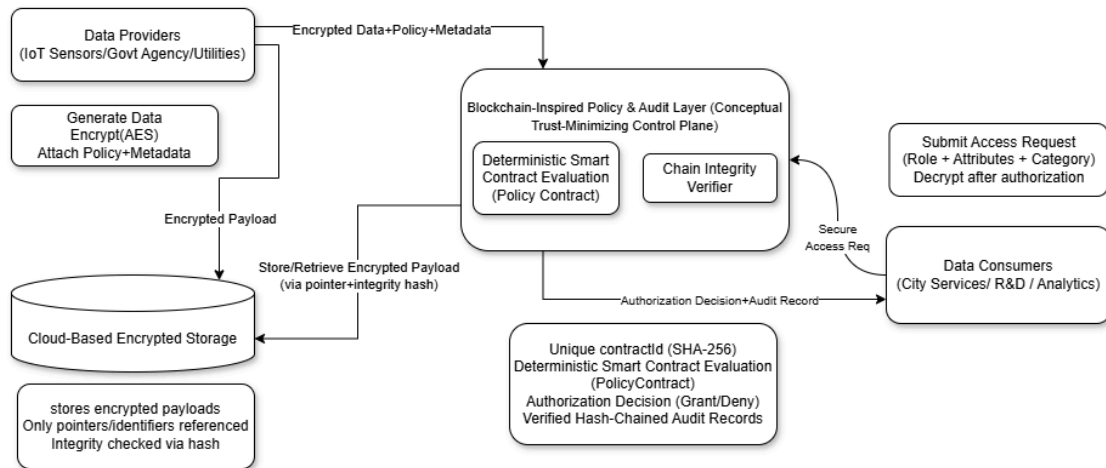


Figure 3.1: DataTrust_SC Conceptual System Architecture

Figure 3.1 depicts the abstract architecture of the DataTrust_SC system. When a data provider is ready to publish a dataset, the system encrypts it, adds a set of access policies and attributes, and stores the encrypted data. Such datasets are uploaded to some off-chain storage, and the reference is put on the smart contracts. The data consumer makes the request based on their role and attribute. A blockchain-inspired policy and audit layer processes it based on RBAC/ABAC-style rules, validates the dataset by its hash, and puts the result of verification into the structured audit log. In the implemented prototype, these processes are replaced with backend policy check and hash-chained audit logging, respectively.

3.2 THREAT MODEL

For the Privacy-aware Data-sharing model to be implemented, a concrete Threat Model must be defined. Three major types of adversary are assumed in the system (DataTrust_SC): The Honest-but-Curious user, Malicious Attackers, and the Collusion between entities.

The Honest-but-Curious user is a real participant that would play by the protocol's rules but would attempt to gain some insight about the extra sensitive information that might not be explicit from the accessible dataset. Some examples of such malicious behavior include behavioural inference, and correlation of attributes using accessible information. The model handles this with the application of encryption, the Enforcement of access policy, and MinimalDisclosure, respectively.

Malicious Attackers can be external or internal. External attackers aim to achieve unauthorized access and manipulation of data or submitting invalid requests to cause the system disruption. Attackers can even be those that want to evade the backend policy control or the manipulation of audit records. This risk has been addressed by the system in multiple ways: AES based encryption of data storage, authorization checks done in the backend before accessing the data, dataset hash reference for checking the integrity of data and logging of each successful or denied access in the system audit records.

Collision occurs when entities collaborate together in making requests so as to either gain access to restricted data or to subvert access controls. This threat is addressed by having a policy enforcement for every access request and logging all the decisions in the audit log. Advanced forms of collusion are not addressed.

Another consideration is audit manipulation, as the system prototype uses MongoDB rather than a distributed blockchain ledger, which provides a truly auditable system in the long term. Nevertheless, the Hash-Chain features and a simple hash-based checkable detection of an altered or broken Audit history within the simulation framework.

Table 3.1: Threat Model Summary

Threat Type	Description	Potential Impact
Honest-but-Curious	Follows protocol but attempts inference	Privacy leakage, attribute inference
Malicious Attackers	External or insider adversaries	Unauthorized access, data tampering
Collusion	Multiple entities cooperate maliciously	Policy bypass, aggregated inference
Audit Manipulation	Attempts to modify or reorder audit records	Loss of accountability

This threat model also leads to the security design that includes encryption schemes, backend policy check and audit trails in DataTrust_SC.

3.3 DESIGN REQUIREMENTS

Security, privacy, auditability, and performance requirements must be fulfilled by DataTrust_SC, which allows it to reliably support a smart city simulation.

Security requirements comprise the integrity of stored data, an accurate authorization mechanism, authentication of requests, integrity reference and logging of sensitive events. DataTrust_SC encrypts the stored data, and the backend enforces policies of data trusts to only permit intended users to access datasets. Hash references are implemented to verify data integrity, and audit entries keep the record to enable tracing.

Privacy requirements ensure the confidentiality of sensitive data, and are enforced by the following: AES-based encryption, role- and attribute-based access control (RBAC/ABAC), minimum disclosure and controlled decryption. Only data appropriate for the authorized role, attribute and policy conditions of a given dataset may be accessed by a user.

Auditability requirements ensure the completeness of all important access activity records. In the prototype, this is achieved by enhancing the requirement with hash-chained audit logs. PreviousHash, entryHash, chainIndex, and contractId fields in the entries enable ordering and verifying of the audit chain.

Performance requirements ensure the predictable processing of access requests. Because this prototype relies on a simulation environment, it is acceptable that authorization latencies and transaction costs are represented by simulation values, not measurements taken from a real blockchain.

Table 3.2: Design Requirements Summary

Requirement Type	Description	Mechanism
Security	Data confidentiality, access control, integrity reference, auditability	AES encryption, backend policy checks, dataset hashes, audit logs
Privacy	Minimal disclosure and policy-based data access	RBAC/ABAC-style authorization and controlled decryption
Auditability	Traceable grant/deny decisions	Structured audit records with timestamps and dataset hashes
Tamper Evidence	Detectable broken audit history	previousHash, entryHash, chainIndex, chain verification
Performance	Predictable simulation behavior	Simulated latency and transaction-cost indicators

3.4 ASSUMPTIONS

The DataTrust_SC system follows several assumptions that make analysis possible under a controlled simulation setting.

First, cryptographic primitives used in the framework, such as AES-based encryption and SHA-based hashing, are assumed to be correctly implemented by the underlying libraries. This thesis does not propose a new cryptographic algorithm.

Second, communication between data providers, data consumers, and the backend authorization server is assumed to occur over secure channels such as HTTPS/TLS. Network-level attacks and transport-layer failures are not experimentally evaluated.

Third, the conceptual framework adopts blockchain-inspired principles of accountability and trust minimization, but the implemented prototype performs access control and audit logging through backend policy evaluation rather than decentralized consensus.

Fourth, the PolicyContract component is assumed to represent simulated smart-contract-style behavior. It is not a deployed smart contract on a real blockchain platform.

Fifth, system participants are expected to follow protocol specifications except in the adversarial cases described in the threat model. Honest-but-curious users, malicious attackers, and colluding entities are considered conceptually, but advanced automated attack detection is outside the implementation scope.

Finally, the evaluation uses synthetic smart city data. This allows controlled experimentation but does not replace validation with real operational smart city datasets.

3.5 PROBLEM FORMULATION

The main goal of DataTrust_SC is to create a privacy-conscious and trust-minimizing data-sharing model that enforces fine-grained access control while reducing unnecessary disclosure.

Let:

$D = \{d_1, d_2, \dots, d_n\}$ represent the set of datasets stored in the system.

$U = \{u_1, u_2, \dots, u_m\}$ represent the set of users or data consumers.

Each dataset d_j is associated with metadata and an access policy:

$M(d_j) = \{\text{category, owner, sensitivity, policy, dataHash, contractId}\}$

where category identifies the dataset type, owner identifies the provider or owner role, sensitivity indicates the privacy level, policy defines the access rule, dataHash identifies the dataset, and contractId identifies the simulated policy-contract instance when applicable.

Each user u_i has attributes:

$A(u_i) = \{\text{role, attribute}\}$

where role represents the requester type and attribute represents additional request-related conditions.

Access control enforcement is defined as:

$$\begin{aligned} \text{Access}(u_i, d_j) &= \text{Granted, if Policy}(A(u_i), P(d_j)) = \text{true} \\ \text{Access}(u_i, d_j) &= \text{Denied, otherwise} \end{aligned}$$

In the conceptual model, this represents deterministic policy enforcement inspired by smart contracts. In the implemented prototype, the decision is made by backend policy evaluation using simulated PolicyContract logic rather than by a deployed blockchain smart contract.

If access is granted, the system decrypts and returns the permitted data:

$$\text{Return}(u_i, d_j) = \text{Decrypt}(d_j)$$

If access is denied, the system returns an access-denied response and does not expose plaintext data:

$$\text{Return}(u_i, d_j) = \text{Denied}$$

Privacy constraints enforce minimal disclosure:

$$\text{AccessedData}(u_i) \cap \text{SensitiveAttributes} = \text{MinimalDisclosure}$$

This means that users should receive only the information allowed by their role, attributes, and dataset policy.

For auditability, every dataset registration and access decision creates a log entry:

$$L = \{l_1, l_2, \dots, l_k\}$$

Each log entry may contain:

$$l_t = \{\text{type, dataHash, role, attribute, policy, granted, timestamp, contractId, chainIndex, previousHash, entryHash}\}$$

For the audit chain, the relationship between entries is:

$$\text{previousHash}(l_t) = \text{entryHash}(l_{t-1}), t > 1$$

The audit chain is valid iff for all stored previousHash matches their respective entryHash, and for all entryHash equals its corresponding entry. This will ensure a tamper-evident audit verification with a simulated client-server system prototype. A true immutable decentralized blockchain is not provided, as no external verification mechanism is introduced, hence no consensus method is available.

To summarize this chapter, we define the formal problem formulation of the DataTrust_SC system, a blockchain-inspired accountability and encryption-based confidential framework, by using a simulated client-server prototype to realize access control and audit logging.

CHAPTER IV

PROPOSED FRAMEWORK DATATRUST_SC

This chapter describes the suggested DataTrust_SC framework. This privacy-aware, blockchain-inspired framework provides secure sharing of smart city data by applying data encryption, fine-grained policy enforcement, hash-based data identification, and auditing via structured logs.

While DataTrust_SC utilizes principles of blockchain technology and their practical validity, it does not use a fully implemented blockchain in a peer-to-peer fashion, as there is no need to do so to evaluate the framework's potential. Thus, DataTrust_SC cannot be regarded as a running instance of blockchain. The framework does not use blockchain nodes operating in a peer-to-peer manner, the consensus algorithm, cryptocurrency transactions, or smart contract deployment.

This chapter focuses on explaining the framework structure, privacy approach, access control, workflow, simulation representation, prototype limitations, and security features.

4.1 OVERVIEW OF DATATRUST_SC

DataTrust_SC offers the following techniques that help with the privacy-friendly, responsible, and trust-minimized exchange of smart city data. The source of smart city data could include IoT devices, government agencies, traffic surveillance systems, utility meters, and civic services providers. Due to the very high chance of the inclusion of confidential information in smart city data, the objective of DataTrust_SC is to maintain the privacy of the datasets, while at the same time granting the required access to the relevant parties.

The key features of this design include:

1. **Trust Minimization:** Authorization will be based on deterministic policy enforcement rather than discretionary authorization. Trust minimization is achieved through the recording of audit trails where all access authorizations are logged.
2. **Privacy Preservation:** Dataset payloads will be encrypted before being stored on the server. Plaintext access will only be granted upon receiving a request that is authorized.
3. **Accountability and Auditability:** Records of dataset registration, requested access and access approval, and access denial, are logged in the audit trails. The prototype is made more robust with hash-chained audit trail entries.

So, the system balances between data utility and data protection. It ensures that stakeholders in smart cities could collaborate and minimize the risks like unauthorized access, data leak, insufficient traceability, etc and lack of accountability.

4.2 ARCHITECTURE DESIGN

The design of DataTrust_SC is built upon a conceptual framework that resembles a blockchain with data stored offline and encrypted. Big smart city datasets are not placed in a blockchain but encrypted and stored within the database, with the control layer performing policy evaluation, hashing, and auditing.

Architecture consists of four layers:

1. **Data Providers:** these actors provide smart city datasets by uploading them or generating the datasets from their sensors, such as IoT devices, public organizations, traffic cameras, and utility meters.
2. **Data Consumers:** consumers represent actors who ask for access to the dataset. Within the prototype, consumers are modeled via roles such as citizens, researchers, and city authorities.
3. **Policy and Audit Layer:** this layer evaluates the requests for access and logs access decisions. The layer is blockchain-inspired but implemented in the backend.
4. **Encrypted Storage:** this layer stores encrypted data of the dataset and its metadata. MongoDB is chosen as a data storage technology.

4.2.1 POLICY AND AUDIT LAYER

The policy & audit layer represents the main controller module inside the DataTrust_SC architecture. It verifies the access policy and performs the audit log entries.

Policy Enforcement Logic: DataTrust uses both role-based and attribute-based policies. The decision regarding the access request is made through the comparison of the requesting role/attributes against the policy set by the dataset. The above logic could be described by means of the PolicyContract module used in the prototype architecture. Each dataset has its own copy of the policy contract generated dynamically upon upload based on the dataset policy, the owner's role, and the data hash value. This contract-based entity verifies the requesting role and attributes and produces an output.

It is not the actual contract in terms of a blockchain application but just backend processing logic.

Access Audit Logging: Dataset registration, access requests, granting, and denying actions are logged in the access audit log. The hash-chaining mechanism is used in the prototype implementation to provide security. Each access audit log entry contains such elements as previousHash, entryHash, chainIndex, and contractId values.

4.2.2 OFF-CHAIN COMPONENTS

Encrypted Storage: The off-chain storage technique will save both the encrypted data and metadata information of datasets. Using this technique, we can store larger datasets of smart cities without incorporating the plaintext in the blockchain.

Hash Based Data Identification and Indexing: Because of the registration process, every dataset gets a hash that acts as the identifier of the dataset along with the integrity reference. This makes it easier to retrieve the record and enforce security policies without exposing the plaintext content by the use of metadata information and indexing.

Table 4.1: DataTrust_SC Architectural Components

Component	Function	Location	Key Feature
Policy Enforcement Layer	Role- and attribute-based access control	Backend control layer	Deterministic rule-based authorization
Simulated PolicyContract	Contract-like policy evaluation	Backend simulation	Structured grant/deny decision with contract ID
Audit Logging Mechanism	Access event recording	Database audit collection	Traceable and hash-linked records
Encrypted Storage	Dataset payload storage	Off-chain database	AES-based confidentiality
Hash-Based Identification	Integrity reference and indexing	Control reference layer	Dataset and audit hash support
Chain Verification	Audit-chain integrity check	Backend verification endpoint	Detects broken audit-chain links

4.3 PRIVACY PRESERVING MECHANISM

The privacy protection mechanism used in DataTrust_SC includes mechanisms like encryption, policy enforcement, limited disclosure, and audit logging. The privacy protection technique suggested in this paper cannot ensure the privacy of the users against attacks on their private data. It can ensure the privacy of the user only in a simulation environment.

4.3.1 ENCRYPTION STRATEGY

The payload for datasets is first subjected to encryption that utilizes AES-based encryption. This is intended to ensure that plaintext data is not disclosed during storage. After a data provider submits the payload of the dataset, the payload gets encrypted by the backend and then gets saved together with metadata and the access policy of that dataset.

Upon fulfillment of the conditions stipulated in the access policy, the backend then decrypts the dataset and provides access to the decrypted plaintext data.

4.3.2 KEY MANAGEMENT

The management of the key in the suggested approach will be conducted through authorization from the backend. The right of decryption is provided according to the policy associated with the dataset. In this case, neither blockchain-based key management is considered nor are smart-contract-enabled releases of keys utilized; decryption rights will depend upon the backend after execution of policies.

The important feature here is that although it is currently possible to have restricted decryption through simulation, more complicated decentralized key management should be investigated later.

4.3.3 IDENTITY PROTECTION

Attributes and roles of the requester will determine whether the user has access rights to the system. Instead of letting any user have access to any resource available, it will be validated whether the requester qualifies for the particular policy. Therefore, limited disclosure becomes possible since only those datasets for which a user has qualified are decrypted.

The implementation of advanced anonymous credentials and identity proofing is not done in the prototype.

4.4 ACCESS CONTROL MODEL

DataTrust_SC utilizes a mixed-model approach to access control which uses both Role-Based Access Control and Attribute-Based Access Control.

RBAC provides flexibility to make access decisions based on the roles of users such as citizen, researcher, or city official. ABAC provides additional flexibility in making access decisions based on attributes such as dataset characteristics, dataset ownership role, request characteristics, and so forth. Thus, such a hybrid approach is better suited for smart cities data sharing compared to the purely role-based approach.

4.4.1 ROLE-BASED AND ATTRIBUTE-BASED ACCESS

The role-based part of the model represents an initial permission granted based on a user's role. ABAC allows making finer-grained decisions through attribute evaluation, for example, public datasets about the environment or traffic would be accessible to certain roles, whereas other datasets about utilities or citizens would be accessible to fewer roles.

The approach provides for fine-grained authorization, avoiding unnecessary data disclosure, and allows implementing consistent access control for different dataset types.

4.4.2 POLICY ENFORCEMENT LOGIC

The access control policy takes the form of deterministic rule conditions. When requesting access to the dataset, the user sends an access request to the backend that gets the dataset policy and makes an access decision based on the requester's role and attributes.

ALGORITHM 4.1: POLICY-BASED ACCESS VERIFICATION

Input: User (U), Dataset (d)

Output: Access Granted or Access Denied

1. Retrieve requester role and attributes ($A(U)$)
2. Retrieve dataset policy ($P(d)$)
3. Retrieve dataset hash and policy-contract information
4. Execute simulated PolicyContract evaluation
5. If ($A(U)$) satisfies ($P(d)$), grant access
6. Create audit record for granted decision
7. Else, deny access
8. Create audit record for denied decision
9. Link audit entry with hash-chain fields

This algorithm represents blockchain-inspired deterministic enforcement. It does not represent an actual deployed smart contract.

4.5 DATA TRUST_SC FRAMEWORK WORKFLOW

The workflow of the DataTrust_SC prototype follows these steps:

1. **Data Registration:** the provider uploads the data with its metadata and policy.
2. **Encryption:** the backend system encrypts the data in the form of an AES-based data block.
3. **Hash Generation:** it produces a hash for the dataset in order to be able to reference and verify the integrity of the dataset later on.
4. **Simulated Policy Contract Creation:** it produces a backend object similar to PolicyContract by creating a dataset policy, an owner role, and a data hash.
5. **Dataset Storage:** stores the encrypted data with its metadata in the database.
6. **Data Request:** data consumers request the data including some role information and attributes.
7. **Authorization:** the backend system tries to authorize the data consumers by checking the given role information and attributes with the PolicyContract-like object.
8. **Audit Logging:** the result is either a grant or a deny operation. The prototype only records certain hash information from the audit chain: previousHash, entryHash, chainIndex and contractId.
9. **Data Retrieval:** the data is either sent back or nothing is sent back to the data consumers.
10. **Chain Verification:** data consumers can verify whether the entire log chain remains unbroken.

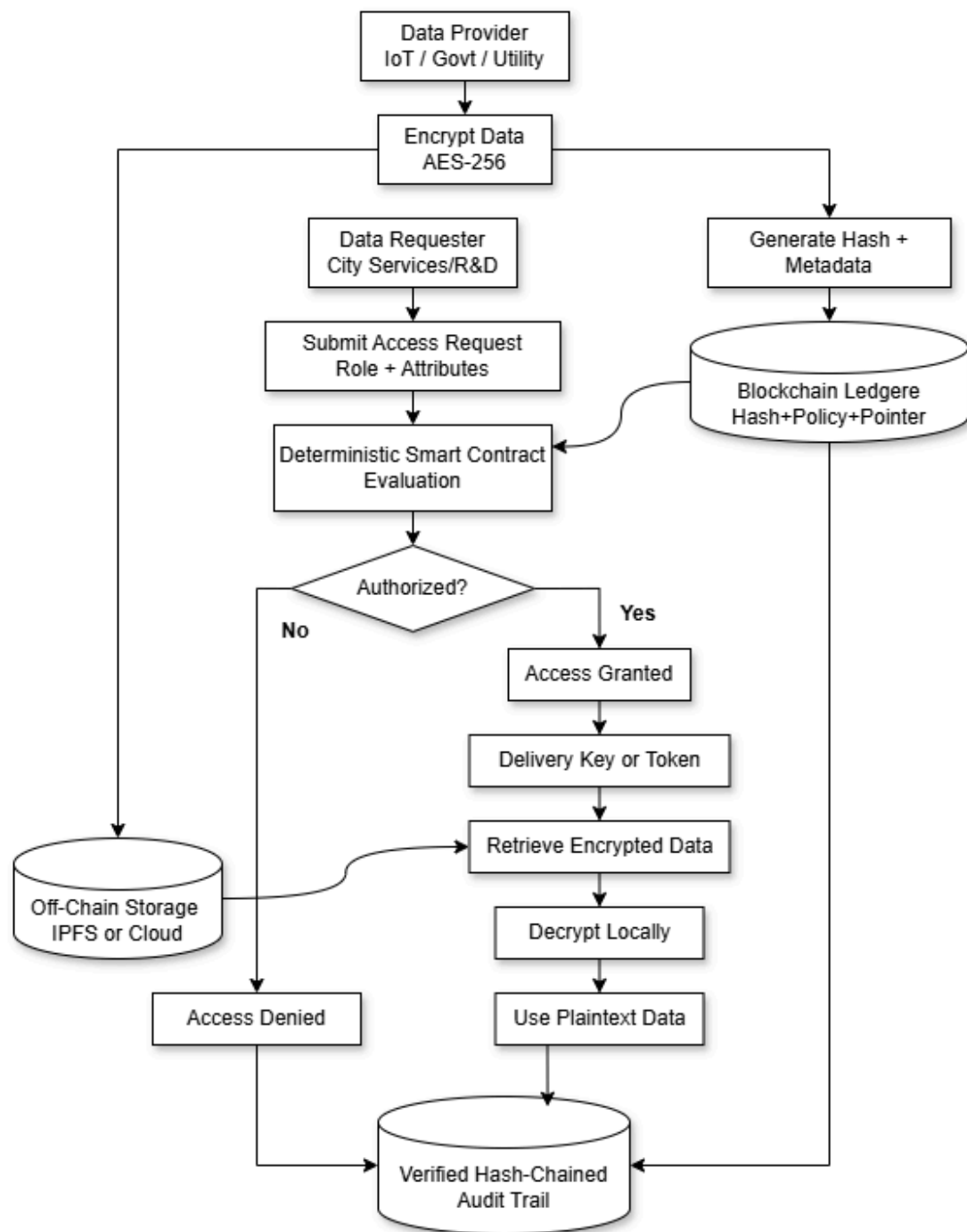


Figure 4.1: Proposed DataTrust_SC Conceptual Workflow

The above conceptual workflow diagram outlines that data providers would encrypt their datasets and create their hashes, along with associated metadata. They then store the encrypted data off-chain, accept access requests and check the policy rules. Based on this, they allow or reject access, and then log this event in the audit trail.

4.6 SIMULATION PROTOTYPE REPRESENTATION

Although the conceptual model presented is a blockchain one, the actual system has been developed as a simulation prototype. The prototype is developed to test the main logic behind the designed system without implementing an actual distributed ledger.

The prototype simulates:

1. Policy enforcement logic
2. Hash generation
3. Encrypted dataset storage
4. Simulated smart-contract-style authorization
5. Grant/deny decision logging
6. Hash-chained audit trail maintenance
7. Chain integrity verification
8. Disaster monitoring and alert display

In the simulated system, there is no distributed ledger. A backend server and organized MongoDB collections are used instead, to mimic the traceability features of a blockchain. The backend is responsible for carrying out the encryption, hashing, policy checks, decision making, logging, verification etc. The frontend, on the other hand, provides facilities to the users, to upload data, send requests, view audit trails, check integrity and observe the dashboard features.

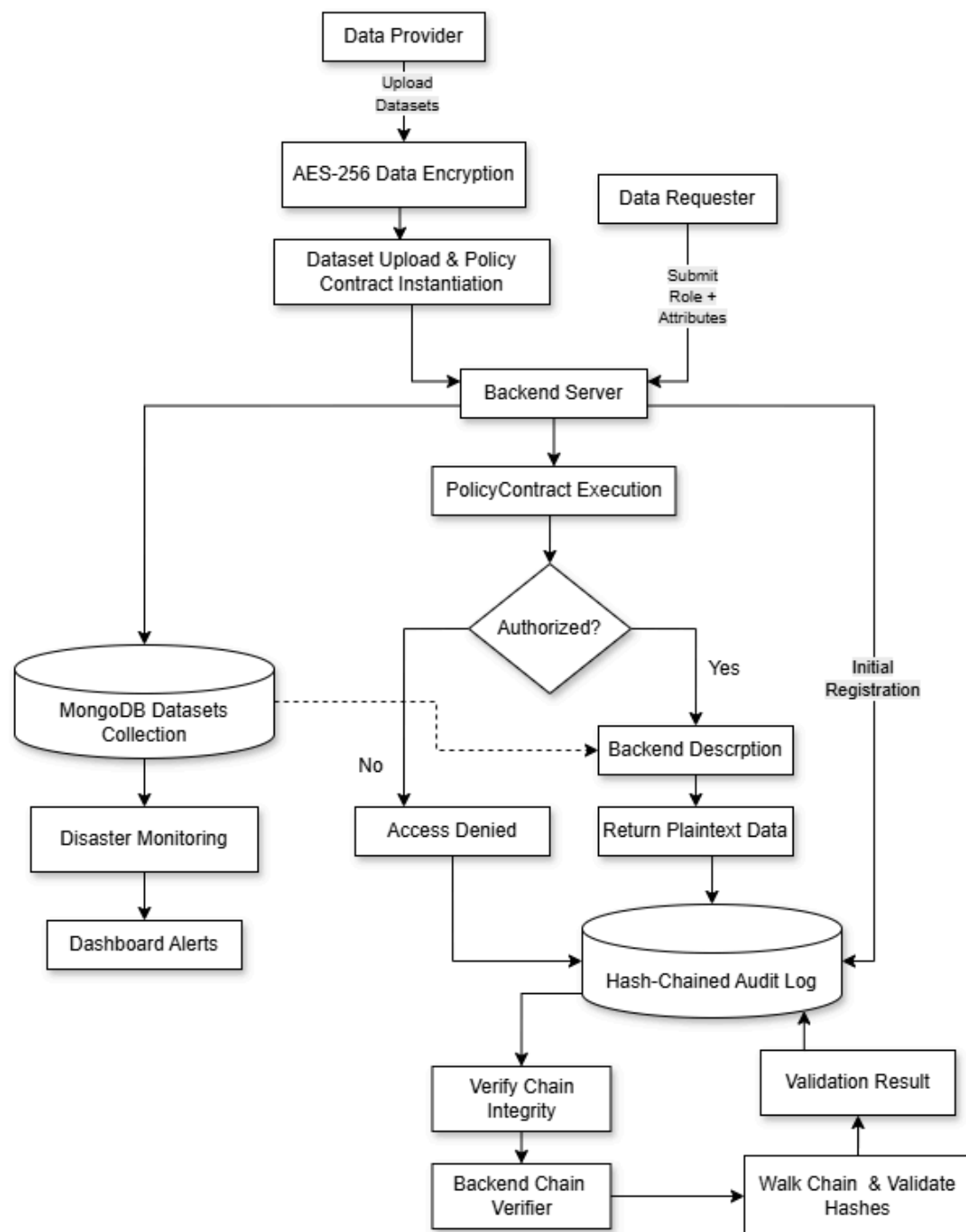


Figure 4.2: Simulation Prototype Workflow

The simulation workflow illustrates the implemented client-server workflow: Uploaded data is encrypted and saved in MongoDB, requested by users, processed by backend policy logic and decrypted after authorization, stored in the audit log.

4.7 PROPOSED FRAMEWORK VS SIMULATION PROTOTYPE

While this prototype simulation does model the DataTrust_SC framework, there is a stark contrast between this conceptual model and the system.

Table 4.2: Proposed Framework vs Simulation Prototype

Aspect	Proposed Framework	Simulation Prototype
Enforcement Model	Blockchain-inspired policy and audit layer	Backend policy evaluation module
Smart Contracts	Conceptual deterministic rule encoding	Simulated through PolicyContract backend logic
Ledger	Conceptual immutable audit layer	MongoDB-based hash-chained audit log
Audit Chain	Tamper-evident record sequence	previousHash, entryHash, chainIndex, contractId
Storage	Encrypted off-chain repository	Encrypted database storage
Hash Integrity	Cryptographic dataset and log identifiers	Hashes generated and stored in backend
Decentralization	Trust-minimizing conceptual design	Centralized backend simulation
Consensus	Required only for future real blockchain deployment	Not implemented
Deployment	Theoretical architecture	Web-based client-server prototype

This table better illustrates the extent of this contribution. The prototype simulates blockchain-like actions, but does not implement blockchain-level decentralization.

4.8 SECURITY FEATURES

DataTrust_SC implements the following security controls within the confines of the simulation prototype.

1. **Confidentiality:** Dataset data is encrypted before storage. When data is returned from the repository, it is done so in plaintext only after authorization.
2. **Integrity:** Hashes of datasets are available which can act as a reference for data integrity. Chained hashes are used for auditable records which may aid in detecting a break in the audit history.
3. **Authorization control:** RBAC/ABAC style policy evaluation controls who can read/write what datasets.

4. **Auditability:** All access requests and authorization decisions are recorded in structured audit logs.
5. **Tamper-evident audit logging:** The audit logs in the prototype include chaining which relies on the previousHash, entryHash, and the chainIndex value. A chain verification check is done which ensures that the log entries are in order.
6. **System-level accountability and non-repudiation:** The requester information (as available within the prototype's context), time stamps of requests, data sets being requested, the policies being consulted, and the decisions to grant or deny access are logged within the prototype's context and provide an auditable history. The prototype does not perform blockchain digital signatures, therefore not full cryptographic non-repudiation.

Table 4.3: Security Features of DataTrust_SC

Security Feature	Mechanism	Benefit
Confidentiality	AES-based encryption	Protects dataset content
Integrity	Cryptographic hashing	Supports tamper detection
Authorization Control	RBAC + ABAC policy logic	Enables fine-grained access
Simulated Contract Enforcement	Backend PolicyContract	Provides deterministic policy decisions
Auditability	Structured audit logs	Creates traceable access history
Tamper Evidence	Hash-chained audit records	Detects broken audit-chain links
Accountability	Logged grant/deny decisions	Supports review of user actions

In this chapter we presented the proposed DataTrust_SC framework. It was based on the integration of encryption, fine-grained policy enforcement, hash-based dataset identifiers, and structured logs that allow the sharing of smart city data with privacy guarantees.

The differences between the conceptual design, inspired by the blockchain, and the implemented simulation prototype were emphasized. The design of the former is based on core principles of blockchain technology: deterministic authorization, traceability, tamper-evident logging, and trust minimization which have been represented in the simulation by the presence of policy evaluation on the backend, simulated PolicyContract methods, encryption of storage by MongoDB, chain-verification of hashes, and hash-chained logs. No real blockchain is proposed, no decentralized consensus mechanism is proposed, no deployed smart contracts are suggested. The work demonstrated that there is a way of mimicking the principles of blockchain using a prototype data-sharing system of a smart city. The 5th chapter deals with the implementation and the experiment setup.

CHAPTER V

IMPLEMENTATION AND EXPERIMENTAL SETUP

This chapter describes the implementation and experimental configuration of the DataTrust_SC simulation prototype. The prototype was developed to demonstrate the main functions of the proposed framework described in Chapter 4 within a controlled smart city data-sharing environment. The implemented system demonstrates dataset registration, AES-based encryption, policy-based access control, simulated smart-contract-style authorization, structured audit logging, and disaster monitoring.

The implementation must be understood as a simulation prototype rather than a full decentralized blockchain deployment. It does not implement peer-to-peer blockchain nodes, real consensus, cryptocurrency transactions, or deployed blockchain smart contracts. Instead, selected blockchain-inspired properties are simulated through backend policy logic, a simulated PolicyContract component, hash-based dataset identifiers, hash-chained audit records, and audit-chain verification. This distinction ensures that the prototype scope is represented accurately without overstating it as a real blockchain system.

The implementation is intended to test the behavior of the system in handling datasets, enforcing access policies, recording access decisions, and producing simulated performance indicators under smart city-inspired workloads.

5.1 IMPLEMENTATION ENVIRONMENT

The DataTrust_SC prototype was developed as a Web interface and allows a moderated data exchange between smart city data providers and data consumers. The three-layer architecture consists of the front end (user interface), the back-end processing (application server), and data storage (database).

The image shows a simulated prototype of the DataTrust-SC interface, which is a Trusted and Privacy-Preserving Data Distribution Framework for Smart Cities. The interface is divided into three main panels:

- Data Owner Upload:** This panel includes fields for Owner Role, Sector, Provider Type, and Category. It also has an optional section for Sensor Data (Temperature, Air Quality Index, Traffic Density, Energy Consumption) and an Access Policy section with radio buttons for Predefined Policy and Custom Policy, and a dropdown for Select Access Policy. An Upload Data button is at the bottom.
- Data Requester:** This panel includes dropdowns for Select Role, Select Attribute, and Select Category, followed by a Request Access button.
- Blockchain Audit Log:** This panel shows a Filter by Type dropdown (set to All Logs), a Verify Chain Integrity button, and a message stating "No logs available yet".

Figure 5.1: Simulated Prototype

Figure 5.1 displays the main interface elements, such as the data owner upload, the data requester panel, and the blockchain audit log panel. Data owner uploads datasets and attaches policies, data requester sends access requests and audit logs are visualized system information including authorising the access or rejecting.

5.1.1 SYSTEM ARCHITECTURE OVERVIEW

The client-server architecture of the prototype is as shown in the diagram below: The frontend is responsible for user interactions and data visualization whereas the backend performs dataset processing, encryption, policy assessment, audit log generation, and disaster surveillance. The database stores the encrypted data and logs generated through the blockchain-based audit mechanism.

The main components of the architecture are as listed:

1. **Frontend Layer:** This layer consists of a user interface which is designed using React. This interface is used for uploading data, requesting data access, viewing the logs of data access, inspecting analytics, verifying audit-chain integrity, and obtaining disaster warnings.
2. **Backend Layer:** This layer uses Node.js and Express.js for the purpose of registering the dataset, encryption, producing a hash for data, evaluating policies, and simulating PolicyContract, approving/rejecting access requests, and logging the audit transactions and verification of the chain.

3. **Database Layer:** MongoDB is used for storing encrypted datasets, their metadata, policies, simulated blockchain-based audit logs, and disaster alert information.

This setup creates a confined experimental environment that is relatively easy to reproduce and test, yet sufficient for illustrating the concepts for smart city data sharing, preserving privacy, and is inspired by blockchain technology.

5.1.2 SOFTWARE AND TOOLS USED

The prototype has been developed using the technologies described below:

1. **React.js:** used to develop the web application UI, for uploading datasets, making access requests, looking at logs, results, and alerts.
2. **Node.js and Express.js:** used to develop back-end API services like managing datasets, encryption and policy evaluation, making requests and storing logs.
3. **MongoDB:** used as a central store for persisting encrypted datasets, metadata about datasets, policy, and logs as well as simulating a chain of records in a blockchain manner.
4. **AES Encryption:** used to encrypt the payload of the datasets to ensure data privacy in the system under simulation.
5. **SHA-Based Hashing:** used to provide a dataset identifier, a hash of an audit entry, and also to ensure data integrity of logs in a chained manner like in a blockchain system.
6. **Simulated PolicyContract Component:** used in the back-end as a stub component to act like a smart contract for enforcement of policies in the system's access control system with blockchain-like design. This is not an actual smart contract.
7. **Chain Verification Utility:** used to ensure the data integrity of the logs by verifying the chain.

The design described above provides a relatively straightforward implementation of the system without requiring an enormous distributed blockchain infrastructure for the simulation.

5.2 PROTOTYPE FUNCTIONAL IMPLEMENTATION

The aim of the prototype was to imitate the core functionalities of the DataTrust_SC system. Modules which are implemented: registration of dataset and encrypting it, enforce the access policies, simulated policy-contract authorization, blockchain inspired audit log, chain validation and monitoring disaster alerts.

5.2.1 DATASET REGISTRATION AND ENCRYPTION MODULE

When a data provider submits their data via the frontend interface the registration process for the dataset is invoked. The submitted request is then forwarded to the backend, where the metadata and payload for the dataset are handled.

The steps taken when registering the dataset are:

1. The dataset metadata and policy information are received by the backend.
2. The dataset payload is encrypted using symmetric AES-based encryption.
3. A SHA-based hash is produced for the dataset, to serve as an integrity reference for the dataset.
4. The encrypted payload is stored in the MongoDB database along with metadata such as category, sector, owner, provider type, sensitivity, and access policy.
5. A dummy instance of PolicyContract is instantiated, for the dataset, using the policy information pertaining to the dataset, its owner role, and its hash value.
6. An audit entry is then generated and appended to the audit log collection for the registration of the dataset.

This ensures that unencrypted datasets are never stored in the database and associates the relevant policy and hash information for the dataset before any access requests for the data are made.

5.2.2 ACCESS POLICY ENFORCEMENT MODULE

This module is used to decide whether to grant the requester access to a dataset or not. In the prototype, policy is in the RBAC/ABAC style. This indicates that the role of the requester as well as the dataset attribute is evaluated during the access decision phase.

Upon receiving an access request the backend:

1. Retrieves datasets from storage which matches with requested category,
2. Compares the requester role & attributes value with the access policy stored in storage,
3. Evaluates the policy logic as an imitation of the PolicyContract,
4. Grants or rejects the access request.
5. If access is granted, the backend decrypts encrypted content, then returns it to the frontend.
6. If access is denied, an access-denied response will be returned without returning plaintext data.
7. The grant or deny decision is recorded in the audit log.

This simulated PolicyContract enhances the policy enforcement model to create more ordered decisions, yet the system is not actually put onto the blockchain; it is only a simulated function on the backend.

Table 5.1: Access Control Policy Matrix

Category	Access Policy (Logical Rule)	Citizen	Researcher	CityAuthority
Environmental	(Citizen OR CityAuthority OR Researcher) AND sensitivity = public	Access Granted (public only)	Access Granted (public only)	Access Granted (public only)
Utility	(CityAuthority OR Researcher) AND sensitivity = private	Access Denied	Access Granted (private only)	Access Granted (private only)
Citizen Service	(CityAuthority OR Researcher) AND sensitivity = private	Access Denied	Access Granted (private only)	Access Granted (private only)
Traffic	(CityAuthority OR Citizen) AND sensitivity = public	Access Granted (public only)	Access Denied	Access Granted (public only)

This policy matrix shows the access logic used by the simulation. The access depends upon both the role and sensitivity of the dataset.

5.2.3 SIMULATED BLOCKCHAIN AUDIT LOGGING MODULE

One of the primary goals of DataTrust_SC is the transparency and audibility of data-sharing actions. Because the prototype is not employing a real distributed blockchain network, blockchain-like auditability is achieved by means of a structured database logging mechanism.

Audit logs are collected in a MongoDB database for the following types of events:

1. Registration of the dataset.
2. Submission of access request.
3. Granting access.
4. Denying access .

Each audit entry will contain attributes such as the time, the hash of the dataset, the role of the requester, the attribute data used for policy decisions, and the outcome of the policy check (true or false) and decision. In our prototype, the audit log is structured not as a flat list of records, but as a collection of hash-chained records.

Each audit log entry would contain the following attributes:

1. previousHash
2. entryHash
3. chainIndex
4. contractId

previousHash contains the hash of the previous record. EntryHash contains the hash of the current record. ChainIndex provides the index of the log entry within the log chain. The contractId is a reference (in this case, the identifier of the simulated policy contract that makes authorization decisions) of the contract that generated the particular authorization.

This means that the audit log has a blockchain-inspired structure. While it does not guarantee absolute immutability of the log as implemented within MongoDB, it does make it detectable if the history has been altered or corrupted, within the simulation.

5.2.4 CHAIN VERIFICATION MODULE

In this prototype, there is a chain verification function. The chain verification function checks if the audit log sequence is consistent. It gets the audit logs from the storage in chainIndex order, recalculates the hash of the entry, and compares the value of previousHash with the hash value calculated from the previous entry.

The result will be 'The audit sequence is correct' if the chain is intact, otherwise, the broken point in the chain can be indicated when the error occurs. The function is provided in the frontend, named 'Verify Chain Integrity'.

The function supports auditability and makes the blockchain-like audit approach more practical, however, the function still belongs to a centralized simulation environment rather than being a distributed consensus.

5.2.5 LIVE DATA SIMULATION MODULE

Live data simulation module-The prototype features a live data simulation module which allows the simulation of synthetic smart city records to perform testing. This module can be used for testing the designed framework without having actual operational city data. Records across five sectors can be simulated using provider-category mapping, category-based field filtering, automatic policy generation, and simulation of blockchain metrics like gas cost and authorization latency etc.

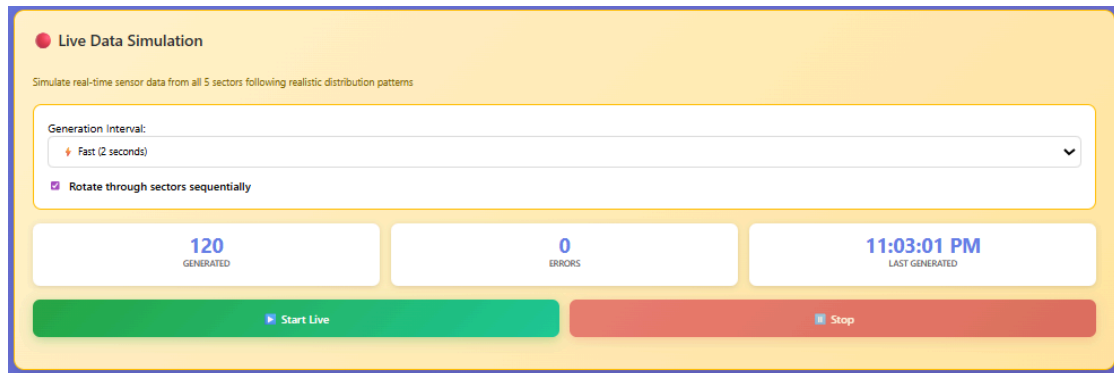


Figure 5.2: Live Data Simulation Configuration Interface

Figure 5.2 is the live data simulation module configuration panel. The user can control the generation interval and enable consecutive rotation of the sector. Also, the number of generated records, the number of errors, last generated time can be seen.

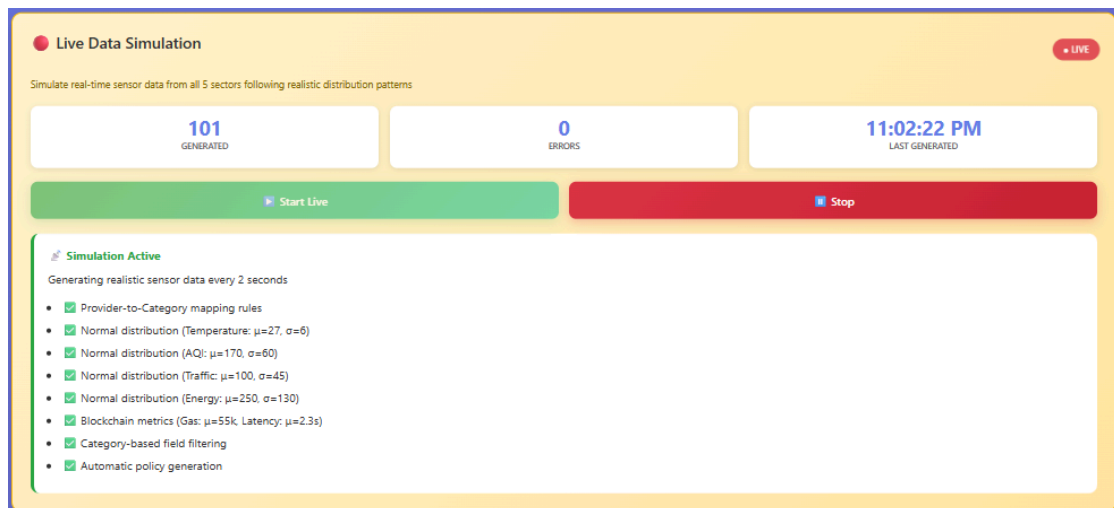


Figure 5.3: Live Data Simulation Active Status Interface

Figure 5.3 demonstrates the module when it is in the running state. Here, the module has confirmed that new synthetic sensor readings were generated every 2 seconds. The list of generation rules that were used is demonstrated as well including normally distributed patterns of temperature, air quality, traffic density, and energy consumption.

This module helps in making the system reproducible, because it allows the prototype to generate reproducible smart-city-type data. Thus new live synthetic data can be produced, to be used in the disaster warning and aid center module.

5.2.6 DISASTER WARNING AND AID CENTER MODULE

The prototype includes a disaster monitoring component to demonstrate a practical smart city application. The module analyzes stored smart city records and generates alerts according to threshold conditions.

The monitored abnormal conditions include:

1. High temperature events
2. High air pollution events
3. Heavy traffic congestion events
4. Abnormal energy consumption patterns

The alerts are displayed on a dashboard interface. This module demonstrates how securely managed smart city data can support decision-making and real-time monitoring. It is not a full emergency management system, but it illustrates a use case for controlled data sharing.

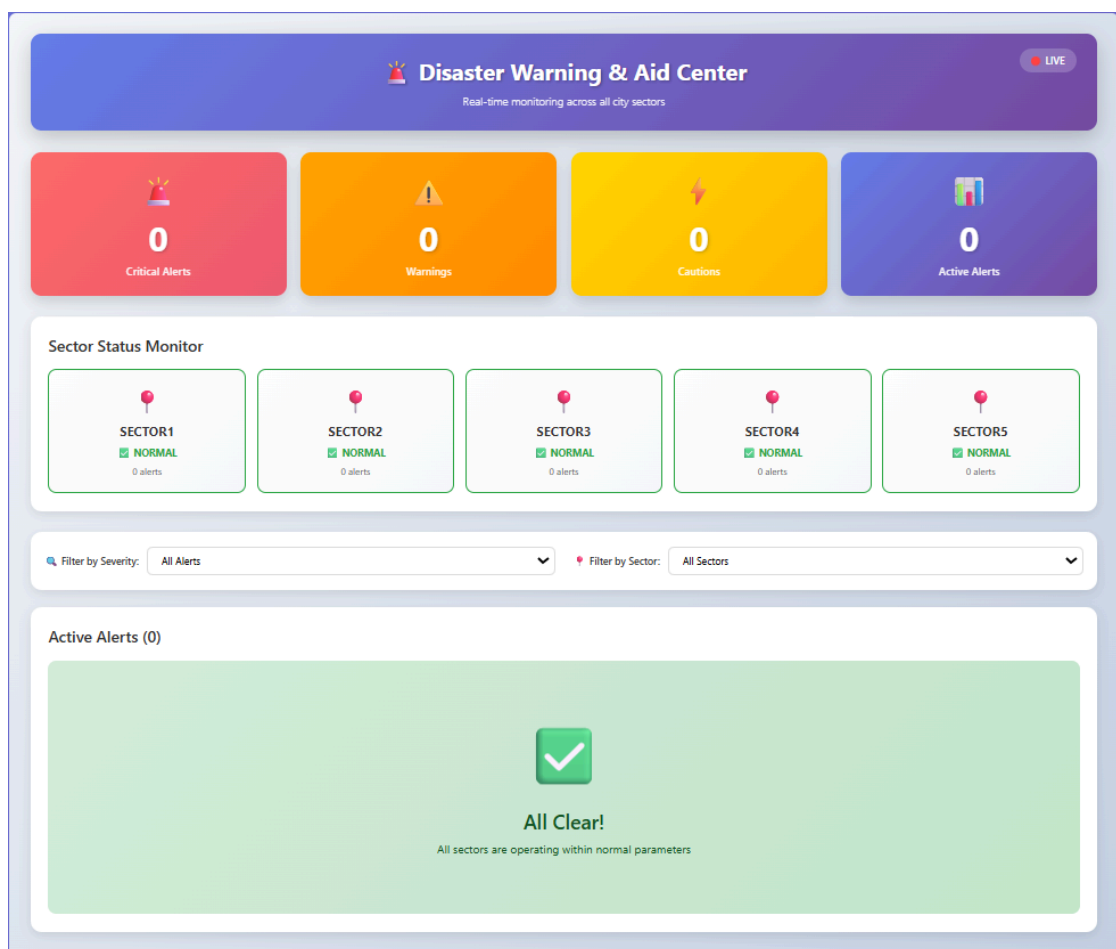


Figure 5.4: Disaster Warning and Aid Center

The disaster dashboard (Figure 5.4) displays critical alerts, warnings, cautions, active alerts, sector status, and filtered alert information. The disaster module uses centralized threshold references instead of scattered hardcoded conditions.

5.3 DATASET DESCRIPTION

The smart city data was generated using a scripted data generation process, which allows the data to represent various heterogeneities of data provider types, categories, sectors, and data measurement fields. The data does not reflect actual operational data in smart cities. While this lack of direct real-world validity is critical, it does enable controlled testing of the prototype.

Four data provider types exist within the data set:

1. IoT Sensor
2. Public Agency
3. Traffic Camera
4. Utility Meter

All the generated records belong to some smart city sector, which ranges from sector 1 to sector 5. The following data categories exist in the data set:

1. Environmental
2. Traffic
3. Utility
4. Citizen Service

Each record belongs to a category, and each category contains unique measurement fields, if applicable. An environmental data record may or may not contain readings on the temperature and air quality within a region, citizen service records may or may not contain records on temperature and air quality, traffic data records may or may not contain records on the current traffic density, and utility records may or may not contain records on energy consumption. All measurement fields will contain null or sample data when not applicable to a given category, to prevent schema variation.

Table 5.2: Dataset Provider and Category Mapping

Data Provider Type	Possible Category	Output Fields	Data Owner
Utility Meter	Utility	Energy_Consumption_ kWh	CityAuthority
Traffic Camera	Traffic	Traffic_Density	CityAuthority
IoT Sensor	Environmental OR Utility	(Env) Temperature_C, Air_Quality_Index OR (Util) Energy_Consumption_ kWh	Citizen OR Researcher
Public Agency	Environmental OR Citizen Service	Temperature_C, Air_Quality_Index	CityAuthority

Each dataset record includes the following attributes:

1. Record_ID
2. Timestamp
3. Sector
4. Data_Provider_Type
5. Data_Owner
6. Data_Category
7. Temperature_C
8. Air_Quality_Index
9. Traffic_Density
10. Energy_Consumption_kWh
11. Blockchain_Tx_Cost_Gas (simulated)
12. Authorization_Latency_sec (simulated)
13. Access_Policy

The payloads of datasets are encrypted with AES encryption before being stored and dataset hashes are produced to represent integrity. The audit logs records also have these hashes as identifiers.

5.4 EXPERIMENTAL SCENARIOS

The prototype of DataTrust_SC was tested for its access control behavior, audit logging behavior, and simulation of its performance behavior by a range of scenarios. The scenarios were defined based on the React front-end, Node.js/Express back-end, MongoDB database, AES encryption, and policy evaluation modules implemented.

The scenarios tested were:

1. ACCESS REQUEST WORKLOAD TESTING

Requests were sent to the system by simulating varying numbers of accesses so that the responsiveness and stability of the system could be analyzed. The tested requests were:

1. 100 requests
2. 500 requests
3. 1000 requests
4. 5000 requests.

By sending these amounts of requests, authorization response behavior and system stability under a given load were evaluated. These are simulation inputs and are not real blockchain network load testing.

2. POLICY-BASED ACCESS VALIDATION TEST

In this test, policy-based access control was evaluated by checking if the system permits/denies requests to access requested data with RBAC/ABAC-style policies. The requests are defined based on different combinations of requester roles, dataset categories, and sensitivity attributes.

This test is to determine if datasets will only be provided to authorized requesters who satisfy the defined policy. Denial requests will be blocked and will appear in the audit log.

3. UNAUTHORIZED ACCESS SIMULATION TEST

In this test, requests with non-corresponding roles or sensitive information are made so as to determine if the unauthorized request is rejected by the system.

The test aims to make sure that unauthorized requests are rejected and that data cannot be decrypted. The event will be logged in the audit log.

4. AUDIT-CHAIN VERIFICATION TEST

In this test, the audit chain of the system is tested to ensure that previousHash, entryHash, and chainIndex values in the audit log data remain connected.

The purpose of this test is to check if the system behaves in a tamper-evident manner within the simulation. It does not imply blockchain's immutability but tests the internal consistency of the audit chain structure implementation.

5.5 EVALUATION METRICS

Several metrics were used to evaluate the function and suitability of the DataTrust_SC simulation prototype.

1. **Policy Enforcement Accuracy:** Tests if the system permits or denies the access request according to the specified RBAC/ABAC policy. The tests were done by comparing the real authorization response with the expected policy output.
2. **Audit Logging Completeness:** Tests if important system events, such as registering a dataset, authorizing a data access, denying a data access, are logged in the audit log collection or not.
3. **Audit-Chain Integrity:** Tests if the hash chained audit entry can maintain the hash consistency among records by checking the values for previousHash, entryHash, and chainIndex.

4. **Simulated Transaction Cost:** As the system does not really do a transaction on the blockchain, the transaction cost is simulated as gas value and recorded into the generated dataset. This metric is a test indicator and not the actual gas fee on a blockchain.
5. **Simulated Authorization Latency:** The authorization latency is simulated as the value recorded in the dataset, it reflects the time for the access decision in the specific smart city simulation and does not relate to latency on a distributed blockchain network.

Table 5.3: Evaluation Metrics Definition

Metric	Description	Measurement Method
Policy Enforcement Accuracy	Correctness of authorization decisions	Comparison with expected RBAC/ABAC policy output
Audit Logging Completeness	Logging correctness of system events	Verification of stored logs in audit collection
Audit-Chain Integrity	Correctness of hash-linked audit entries	Chain verification using hash fields
Simulated Transaction Cost	Estimated blockchain-style transaction cost	Dataset field Blockchain_Tx_Cost_Gas
Simulated Authorization Latency	Simulated time required for access decision	Dataset field Authorization_Latency_sec

These metrics are used to measure the entire DataTrust_SC simulation prototype in policy enforcement, the thoroughness of the audit log, the consistency of the audit chain, the estimated simulated delay for authorization, and the estimated simulated cost of blockchain-like overhead respectively.

This chapter presented the implementation of the DataTrust_SC simulation prototype and experimental environment, where the client is implemented by the React framework, server-side using Node.js/Express, and data storage is MongoDB. In terms of its function, it includes AES encryption, RBAC/ABAC like policy check, policy evaluation in terms of simulated PolicyContract authority, formatted policy chain-based hash linking and chain checking, and disaster detection. Based on these scenarios and measures, this work will give results and performance evaluation of the prototype in Chapter 6.

CHAPTER VI

RESULTS AND PERFORMANCE EVALUATION

This chapter describes the results from the DataTrust_SC simulation prototype. Specifically, the evaluation measures the accuracy of access control enforcement, the correctness of audit logging, its simulated behavioral execution, distribution of datasets, validation of the audit chain, and utility of the disaster monitoring module. It aims to demonstrate that the prototype accurately enforces role and attribute-based access controls, protects encrypted data from inappropriate access, logs grants and denials, and simulates analytics using smart city-inspired workloads.

The findings from this chapter should be framed with the context of the actual scope of the implementation. The DataTrust_SC simulation prototype is not measured as a true decentralized blockchain deployment. The simulation does not employ decentralized consensus, deploy smart contracts, or make any cryptocurrency transactions. There are also no real blockchain gas fees. Rather, the simulation has been analyzed on a web-based simulation where the blockchain behavior has been emulated on the backend via policy logic, simulated PolicyContract execution, hash-based dataset identifier, and hash-chained MongoDB audit logs with integrity checking.

6.1 ACCESS CONTROL AND AUDIT LOG VALIDATION

This is the first phase of the evaluation where access control and audit log recording are measured. The test of the access control is divided into two sections: where authorization succeeded and failed, and was designed to identify whether the simulation correctly granted access and rejected others.

6.1.1 SUCCESSFUL AUTHORIZATION SCENARIO

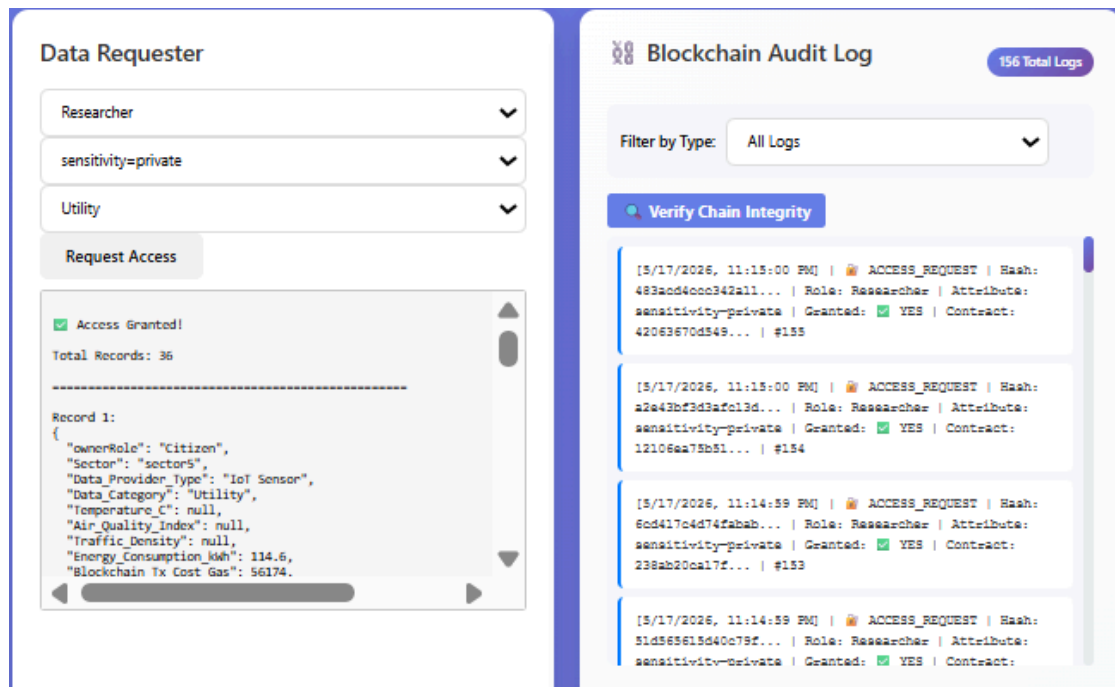


Figure 6.1: Successful Access Request Interface & Audit Log

In the positive authorization case, one researcher submitted an access request for utility category data with attribute sensitivity=private. By the access policy applied in the prototype, researchers are allowed to access private utility datasets. Thus the request is predicted to be authorized.

The outputs of the system include:

1. Access Granted,
2. Total number of records retrieved,
3. Decrypted output dataset.

As shown in figure 6.1, the user is notified about the granted authorization, the records in the dataset are displayed after enforcing the access policy constraint. It shows that the prototype could identify authorized users and show them the desired data only after satisfying policy constraints.

Meanwhile, the access log panel keeps track of access events. The log entry is composed of timestamp, dataset hash, user role, values of attributes passed with the request and access result. In the prototype, log entries may additionally contain values of contractId, chainIndex, previousHash, and entryHash so they can be used to identify simulated policy contracts and store them in a hash-chained audit log.

This proves:

1. The policy rules have been enforced correctly.
2. Decryption happens only after the authorization of users.
3. The authorized access contracts have been logged properly.
4. The design of the audit log allows hash-chained traceability.

6.1.2 DENIED AUTHORIZATION SCENARIO

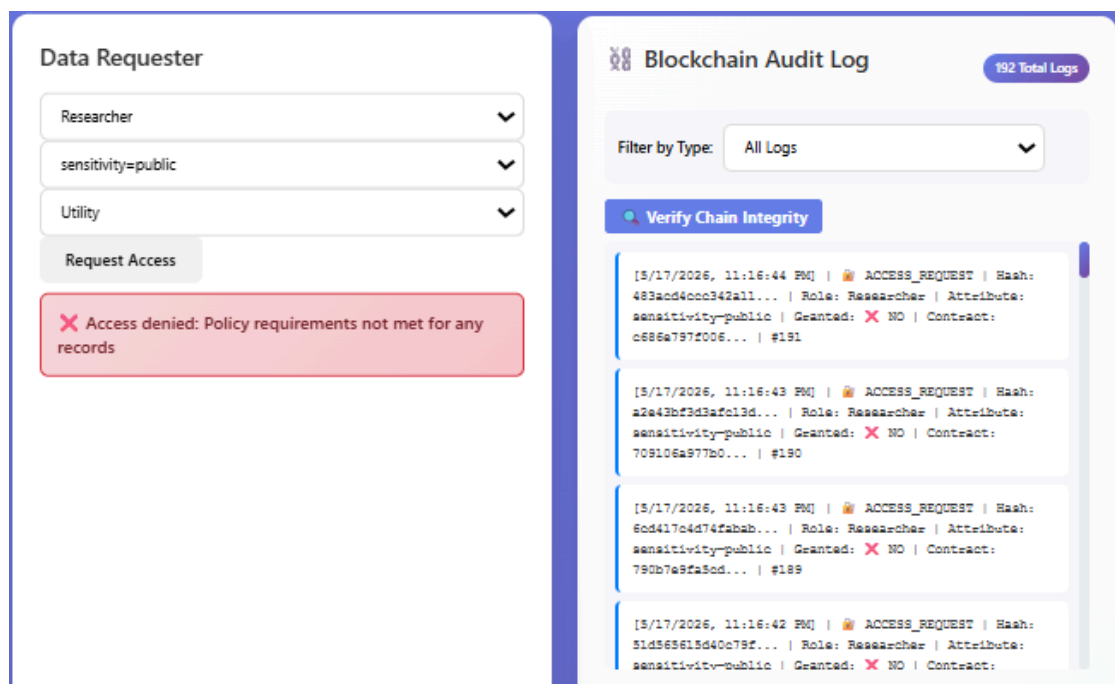


Figure 6.2: Unauthorized Access Attempt Interface & Audit Log

The denied authorization case checks if the prototype blocks a request that does not match the access policy. In this case, the same role researcher was used for requesting data under category utility, and the attribute sensitivity was specified as 'public'. The utility policy stored in the repository was set for private sensitivity for researchers, thus the request was predicted to be denied.

The result recorded was:

1. "Access denied: Policy requirements not met for any records".

As clearly seen in Figure 6.2, the access was successfully denied, and no plaintext data was returned. This confirms that the system does not provide any encrypted information to the user whose role and attribute do not match the policy requirements.

Simultaneously, an entry is also added to the audit log. Timestamp, Role, attribute value, set reference, and authorization status denied. In the prototype, the denied event is also written to the hash-chained audit log. Thus, it also becomes a part of the traceable audit log.

So it's proven that:

1. Unauthorized access attempts are denied,
2. Policy is strictly implemented,
3. Denied events are logged,
4. The audit log contains all access decisions.

All these access-control events, when combined, show that the prototype enforces policy correctly.

6.2 AUDIT-CHAIN INTEGRITY VERIFICATION

Audit logging can be enhanced by transforming the audit log from a flat database collection to a hash-chained collection. Every audit entry may contain previousHash, entryHash, chainIndex and contractId. PreviousHash will link the current entry with the last audit entry, entryHash will store the hash of the current log's data, chainIndex will record the order of the event occurring in the audit log sequence, contractId is to connect access-related logs with the simulation's PolicyContract.

The chain verification function will ensure that the audit chain is valid by reading entries in order by chainIndex, and checking if every entry is pointing to the correct preceding entry.

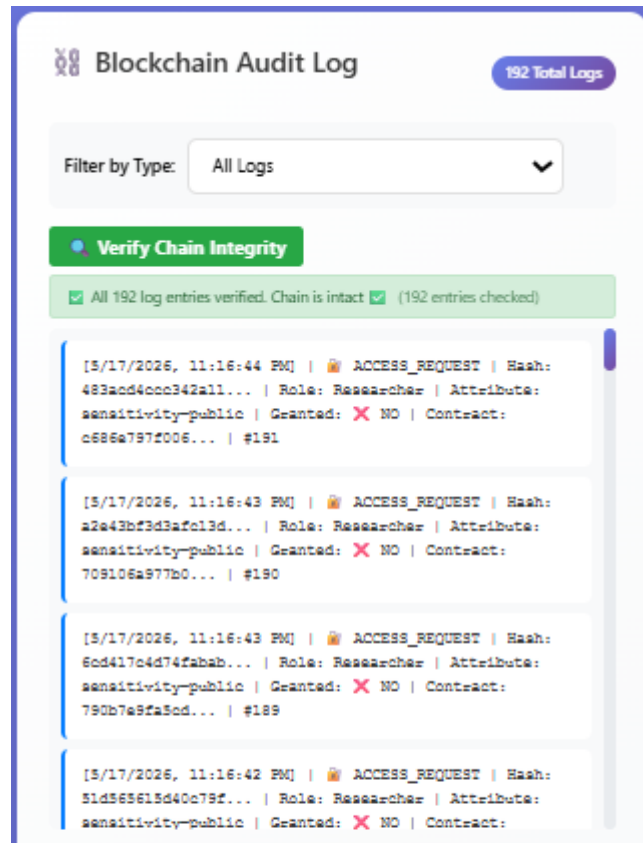


Figure 6.3: Audit-Chain Verification Output When Integrity Is Maintained

Figure 6.3 shows the verification output when audit-chain integrity is maintained, and no broken hash relationship is detected.

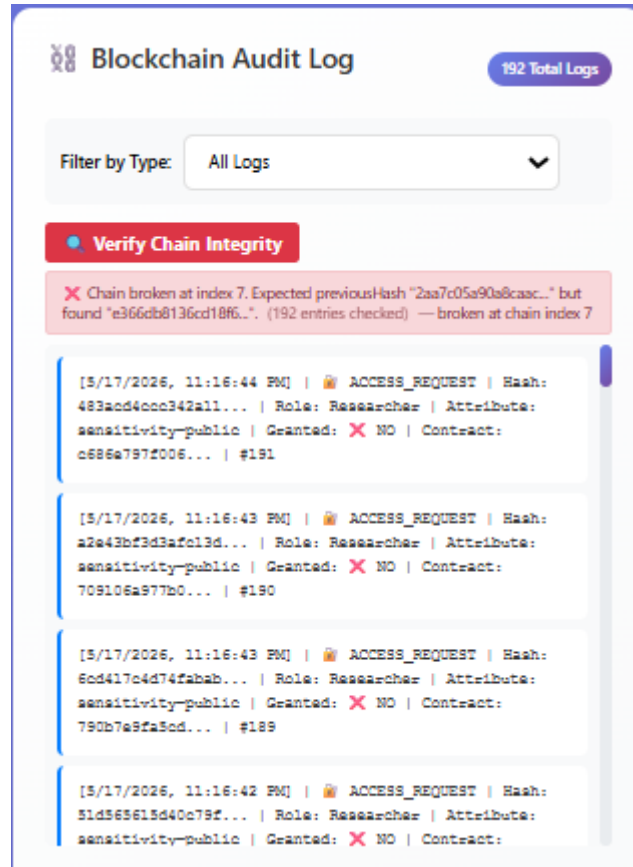


Figure 6.4: Audit-Chain Verification Output When Integrity Is Broken

The verification output when audit-chain integrity is broken is displayed in Figure 6.4. In this case, the system realizes that one or more of the hash links are not correctly connected.

These outputs further reinforce the blockchain-like auditability in the prototype, but they must not be taken as "real" blockchain immutability. The audit chain is stored on MongoDB and is checked from the back-end, therefore providing an unforgeable log file within the simulated environment, and not immutability through distributed consensus.

6.3 SIMULATED PERFORMANCE ANALYSIS

In the second phase of the evaluation, the simulated transaction cost and authorization latency are simulated by using the Blockchain Analytics Dashboard. They are new metrics designed to evaluate how the prototype acts within an ideal smart city data system. They are not the true measurements for a blockchain system.

6.3.1 SIMULATED TRANSACTION COST

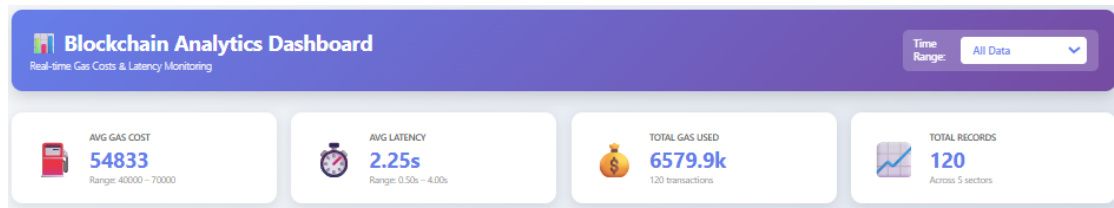


Figure 6.5: Blockchain Analytics Dashboard Summary Metrics

The dashboard summary panel (Figure 6.5) reports:

1. Average Gas Cost: 54,833
2. Defined Range: 40,000-70,000
3. Average Latency: 2.25s
4. Total Gas Used: 6579.9k
5. Total Records: 120

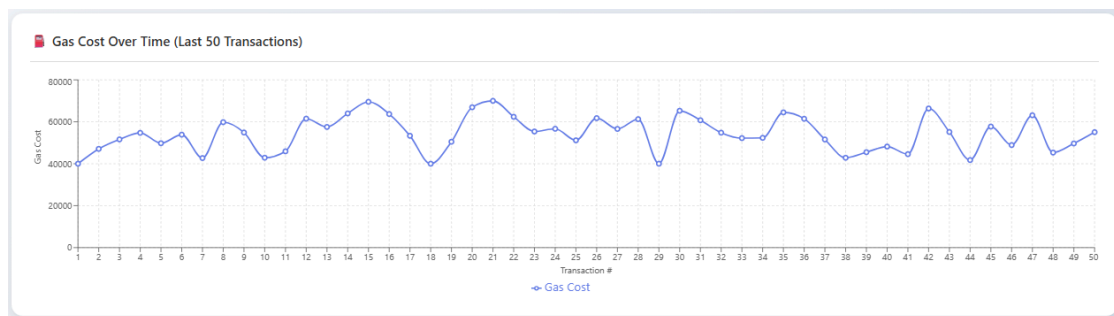


Figure 6.6: Simulated Gas Cost Over Time

The gas cost over time visualization (Figure 6.6) shows moderate variation within the defined simulation range. There are no sharp spikes or abnormal values in the displayed transaction-cost pattern.

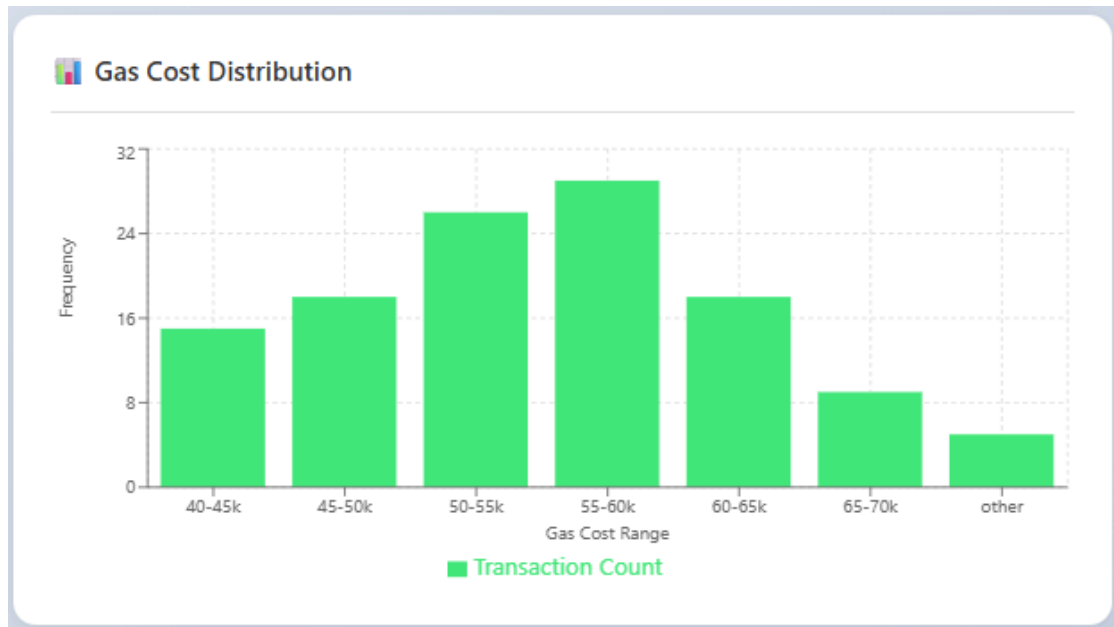


Figure 6.7: Simulated Gas Cost Distribution Across Transaction Ranges

The gas-cost distribution diagram (Figure 6.7) confirms that a majority of generated transactions belong to the range 50k-60k. Records that lie outside the range are labelled as other. This leads us to believe that the behavior of the transaction cost is not particularly inconsistent for simulated data.

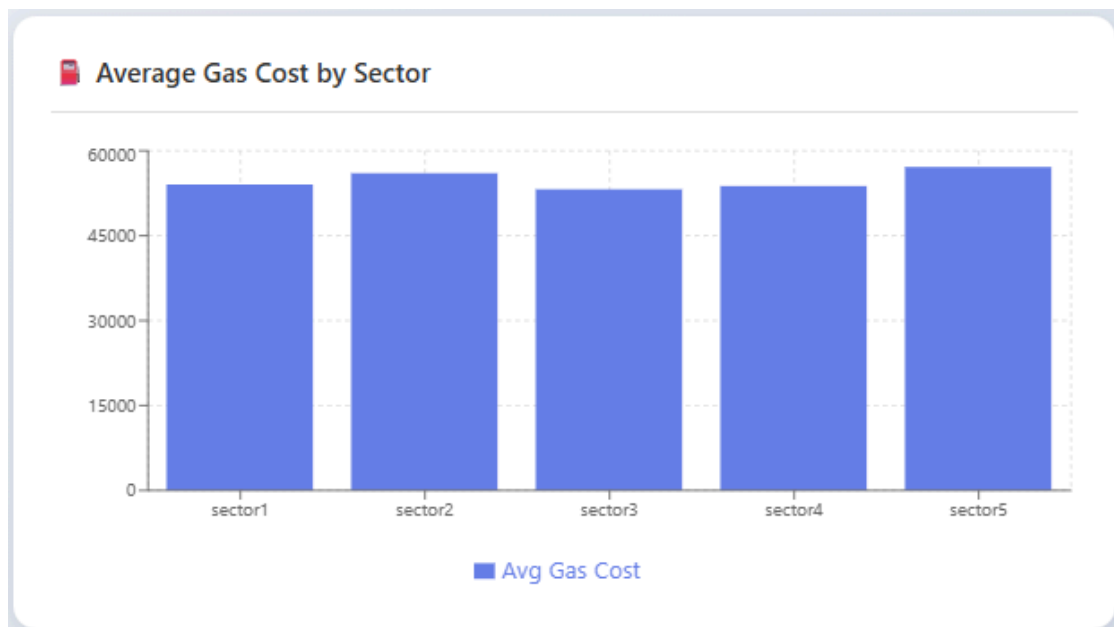


Figure 6.8: Simulated Average Gas Cost By Sector

This chart illustrates the average cost by sector (Figure 6.8), where all five smart city sectors have fairly close simulated values for the transaction cost. So the created dataset can be considered as balanced enough to evaluate the prototype in a controlled environment.

6.3.2 SIMULATED AUTHORIZATION LATENCY

It can be observed that there is a reasonable simulated cost pattern. But still, there are no real blockchain gas fee implications since there is no real blockchain network deployed. The following statistics were presented in the dashboard:

1. Mean Authorization Latency: 2.25s
2. Defined Range: 0.50s-4.00s

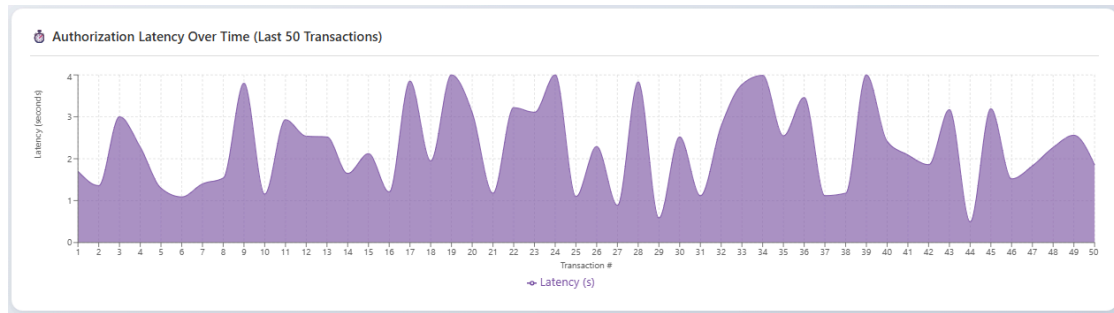


Figure 6.9: Authorization Latency Over Time

The authorization latency over time chart (Figure 6.9) shows bounded variation in the latest transactions. The values remain within the defined simulation range, which suggests that the authorization behavior is stable in the controlled prototype environment.

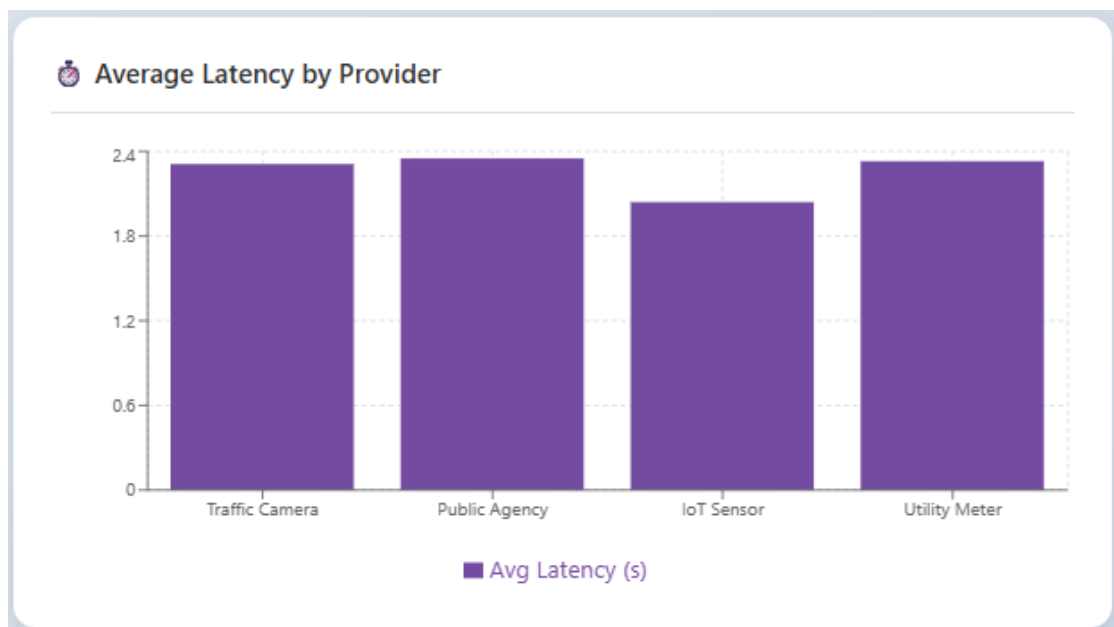


Figure 6.10: Average Authorization Latency by Data Provider Type

The average latency by provider chart (Figure 6.10) compares latency across provider types:

1. Traffic Camera
2. IoT Sensor
3. Public Agency
4. Utility Meter

The chart shows relatively similar latency values across the provider types. This indicates that no single provider type produces abnormal simulated authorization delay.

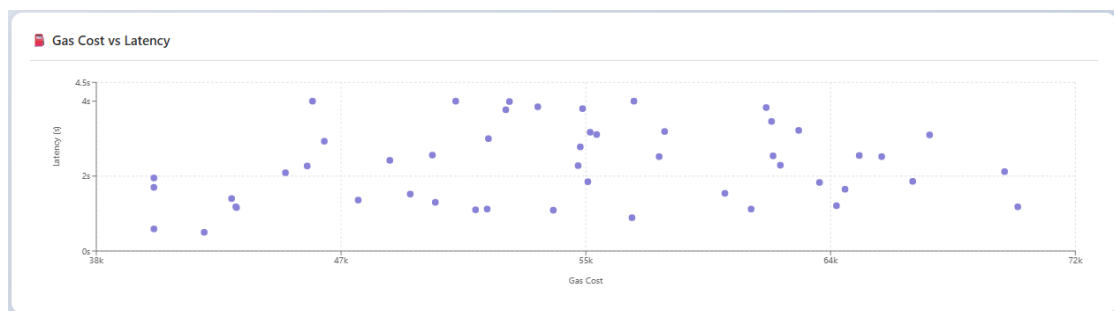


Figure 6.11: Relationship Between Simulated Gas Cost and Authorization Latency

The gas cost vs authorization latency scatter plot(Figure 6.11) shows no correlation between the simulated gas cost and the authorization latency.

It is concluded that:

1. The authorization latency is within the range of the simulation.
2. It does not correspond to an unreasonable latency for Your provider type.
3. The performance shown in simulation is similar regardless of the dataset categories.
4. It shows a prototype-level measurement and should not be considered as a true blockchain network latency.

6.4 DATASET DISTRIBUTION ANALYSIS

The analysis of the distribution of the dataset evaluates whether the synthetically generated smart city dataset is uniformly distributed across the sectors. A uniform distribution of dataset across sectors leads to reduced bias in the results since they do not predominantly rely on any single sector or provider type.

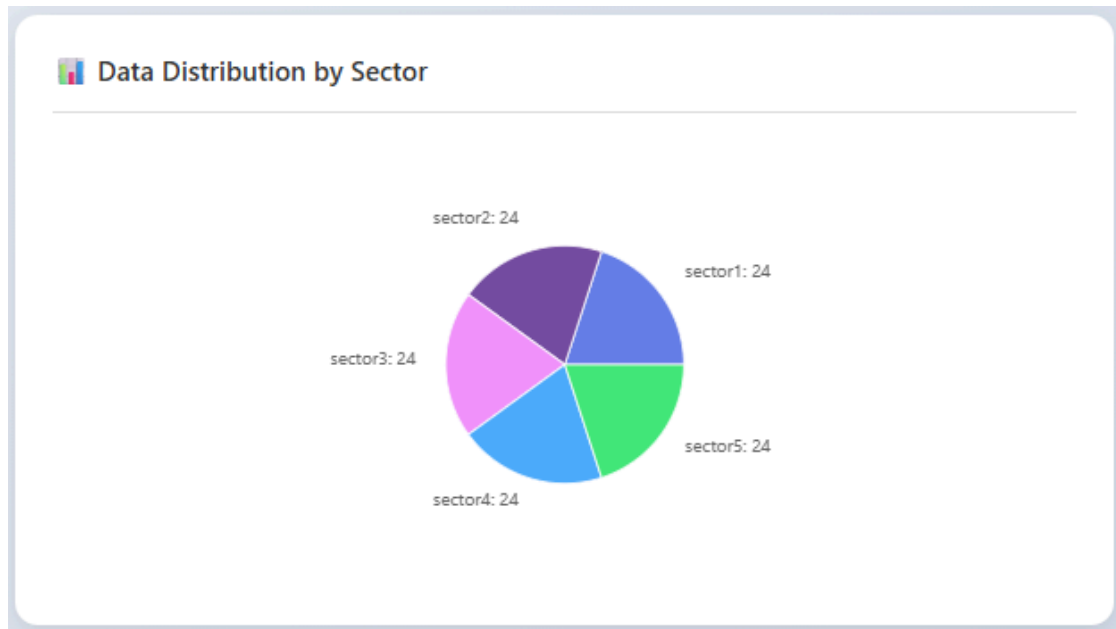


Figure 6.12: Data Distribution by Sector

The data distribution by sector chart (Figure 6.12) shows that the synthetic records are spread across Sector 1 to Sector 5. The visible values indicate that the sectors are represented in a reasonably balanced way. This confirms that the dataset generation logic described in Chapter 5 provides a controlled distribution for testing.

The dataset remains synthetic and should not be presented as real smart city operational data. The distribution is useful for controlled evaluation, but it does not prove real-world validity across live city infrastructure.

6.5 DISASTER MONITORING MODULE EVALUATION

Apart from access control and simulation performance indicators, the prototype also offers the module of the Disaster Warning and Aid Center. This module would analyze the saved smart city data to recognize abnormal situations concerning the environment and infrastructure.

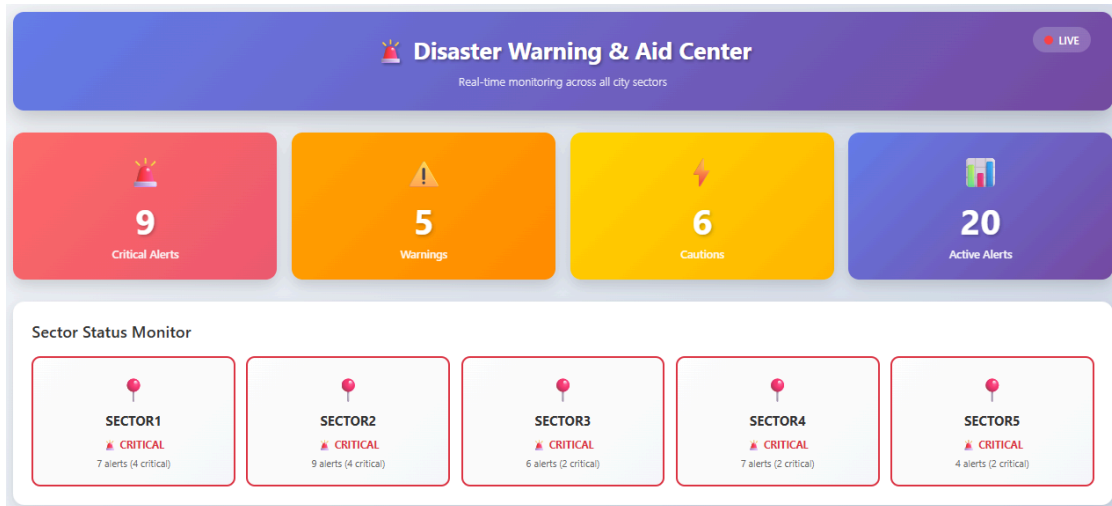


Figure 6.13: Disaster Warning & Aid Center Overview Dashboard

The dashboard (Figure 6.13) displays:

1. 9 Critical Alerts
2. 5 Warnings
3. 6 Cautions
4. 20 Active Alerts

The disaster module analyzes abnormal situations like excessive temperature, unhealthy air quality, extreme congestion and abnormal power usage. The module's threshold logic has also been enhanced and centralized with reference to centrally defined threshold values instead of distributed, hardcoded conditions.

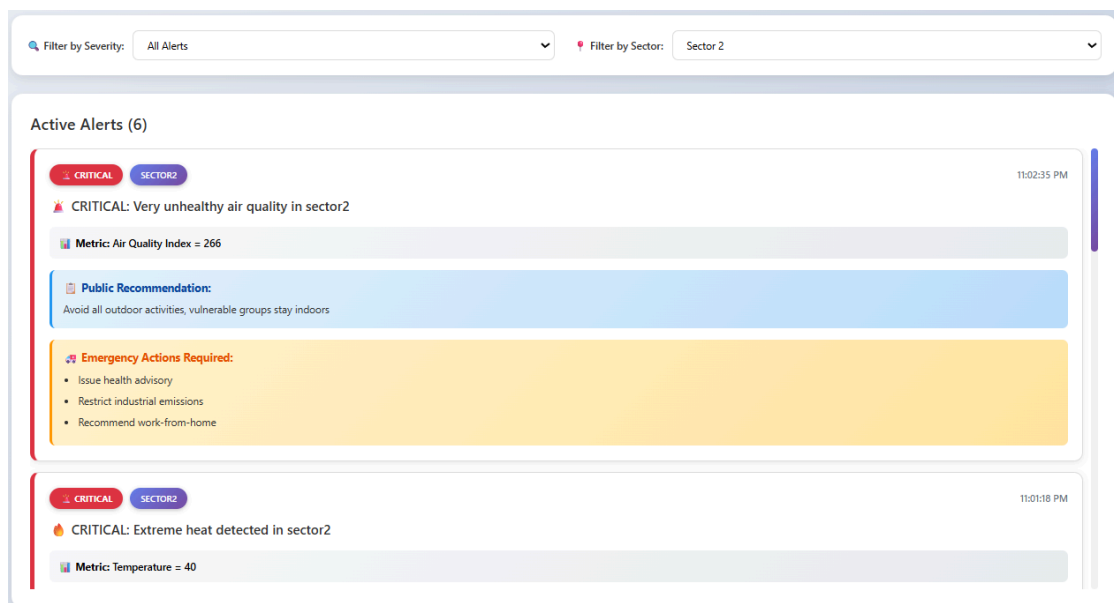


Figure 6.14: Example of Critical Alert with Recommended Actions

An example alert (Figure 6.14) shows:

1. Critical air quality in Sector 2
2. Air Quality Index = 266
3. Guidance, such as avoiding all outdoor activities
4. Emergency action recommendations

This confirms that:

1. Threshold-based detection logic works in the prototype.
2. Sector-level filtering is functional.
3. Alerts are generated from stored dataset values.
4. The module demonstrates a practical smart city use case for controlled data management.

The disaster monitoring module does not represent a complete emergency management system. It is a prototype demonstration showing how governed smart city data can support alert generation and decision-making.

6.6 DISCUSSION OF RESULTS

From the results, we can determine that the DataTrust_SC simulation prototype is capable of enforcing RBAC and ABAC policies, logging authorization decisions, and providing a controlled smart city data-sharing process. Legitimate requests are processed and returned data if policy conditions are met, unauthorized requests are denied and logged. These results uphold the statement that the prototype can be seen as a deterministic policy-based access control model.

The audit logging shows that a specific audit trail is kept, which includes the transaction time, hash of the dataset, role and attributes of the requesting user, and the decision. The decision was made about the request. The chain of previousHash, entryHash and chainIndex allows us to retrieve each recorded entry through its hash chain. ContractId ties each access event to the tested simulation policy-contract.

The simulation's latest performance figures show steady numbers across the various readings, but these need to be put in perspective. Since we're working with a simulation and not a full blockchain deployment, the numbers are based on simulated transaction costs rather than actual on-chain gas costs. The authorization latency is also an indicator of controlled simulation execution rather than real distributed consensus.

The dataset distribution figures indicate that the dummy data that we produced is distributed over a good number of sectors, so we can test thoroughly, but this still indicates it cannot be taken as a definite picture of a real-world scenario.

The results of the disaster monitoring experiment demonstrate that our stored data can be used to produce an output of alerts based on their severity level, which can be used

as a practical benefit of the framework as a data governance and decision support tool prototype.

In summary, the results have shown that the DataTrust_SC can be considered as a feasible prototype within the confines of a blockchain-inspired and privacy-aware simulation. It provides proof that one can perform actions with the aforementioned system including encrypted data handling, policy based authorization and hash chained auditability along with simulated performance analysis and event monitoring for disaster alerting. However these figures only confirm real blockchain performance on a miniature level and there is still room to be tested for large scale use and production security.

This chapter has been concerned with evaluating the DataTrust_SC simulation prototype. The access control tests proved that the properly authorized users can fetch authorized, decrypted records while denying access to unauthorized ones. The audit log verification proved that grant and deny events can be logged by the system. The audit chain mechanism enhances traceability through the creation of hash-linked audit entries and through its verification process.

The simulation result for the performance analysis proves stable parameters of transaction cost and latency within boundaries, and the dataset distribution analysis shows that the synthetic records are partitioned among several sectors, while the disaster monitoring module provided an illustrative application of storing smart city data to generate alerts based on predefined thresholds.

All these results illustrate the ability of the DataTrust_SC prototype as an abstraction to test and simulate in a controlled setting a privacy-aware, blockchain-inspired data sharing mechanism in smart cities. However, all these findings were based on simulation so far and on an experimental prototype; real blockchain, smart contracts with real smart cities data, consensus based audit immutability, actual gas-cost, actual smart city data and the testing in large scale setting should be undertaken as future works.

CHAPTER VII

CONCLUSION AND FUTURE WORK

The final chapter concludes the thesis by giving an overview of the research problem investigated, the proposed DataTrust_SC framework, and the results achieved from implementing the simulation prototype. It highlights the major contributions of this research, addresses the limitations of prototype-based testing, and outlines future directions for the development of data-sharing systems in smart cities that would be secure, privacy-preserving, and accountable. It clarifies that the final contribution is not the implementation of a fully decentralized blockchain network but rather the DataTrust_SC framework tested via simulation.

7.1 CONCLUSION

Contemporary smart cities rely on vast amounts of data gathering and exchange between different organizational entities in order to provide efficient services, monitor infrastructure, plan city development, organize emergency response services, conduct environmental monitoring, and practice evidence-based governance. At the same execution overhead, traditional approaches to data exchange involve sharing via centralized platforms and raise many difficulties. These problems involve unauthorized access to data, privacy violations, insufficient transparency, poor auditability, data tampering risks, and excessive dependency on one authority.

In order to tackle these issues, this thesis proposed the use of DataTrust_SC – the privacy-oriented and blockchain-inspired smart city data-sharing platform. It integrates AES-based encryption, role and attribute-based access control, dataset hashing for identification purposes, simulated smart-contract-style policy evaluation, and structured audit logging. The overall goal of the platform is to facilitate secure data sharing by decreasing superfluous exposure to sensitive information and avoiding reliance on centralized discretionary control mechanisms.

Conceptually, the framework was based on trust minimization, deterministic authorization, privacy protection, and auditability. On the other hand, it should be noted that the implemented solution was explicitly defined as a web-based simulation platform. It is not a real implementation, which means that no blockchain consensus algorithms, peer-to-peer network nodes, cryptocurrency transactions, or even any deployed blockchain smart contracts were employed during the course of developing this prototype.

The prototype demonstrated that it was possible to register a secure dataset using AES encryption and metadata storage. Additionally, it demonstrated that role/attribute-based policies could allow access to authorized users and deny access to unauthorized users. Audit logging was done such that successful and unsuccessful requests could be tracked in the audit logs. The hash-chain method enhanced the audit logging approach, whereby the audit logging was done using previousHash, entryHash, chainIndex, and contractId.

The outcomes indicated that the authorized access requests were granted while unauthorized access requests were denied; audit logs were produced for access attempts made. The simulation of the analytics dashboard gave indicators for the latency for the authorization process and transaction cost. Such values were important when tracking the behavior of the prototype, although they were not measurements of real blockchain systems. The disaster monitoring module demonstrated practical implementation of the smart city use case through data analysis and alerting.

Although the study was successful in achieving its primary objective of demonstrating that the DataTrust_SC architecture is feasible in terms of practical application, it should be noted that the results obtained should be analyzed in the context of simulations and not be regarded as proof of the ability of blockchain to be deployed in the case under discussion.

7.2 CONTRIBUTIONS REVISITED

The contributions made by the current thesis can be classified into conceptual and practical contributions.

7.2.1 CONCEPTUAL CONTRIBUTIONS

Conceptual contribution consists of creating a conceptual design of the system that would enable privacy-preserving data sharing in smart city ecosystems. It involves combining the ideas of confidentiality, fine-grained access control, hash identification, and auditability in one process.

The research contributes to understanding the potential use of trust-minimizing principles in smart city data-sharing systems while admitting that full decentralization has not yet been realized. It offers a structure of data providers, consumers, encrypted storage, access policy, and audit policy.

This thesis additionally helps in making the distinction between the conceptual blockchain design and the implemented simulation. This helps in making the academic precision of the thesis more accurate by not exaggerating the prototype to be a full-fledged blockchain solution but presenting the DataTrust_SC solution as a blockchain-inspired simulation that works on simulating some aspects of the blockchain network in a controlled environment.

7.2.2 PRACTICAL CONTRIBUTIONS

In practice, this thesis helps in creating an experimental simulation prototype for the DataTrust_SC framework, which includes:

1. AES-based encryption of dataset payloads.
2. Role- and attribute-based policy enforcement.
3. Simulated PolicyContract authorization logic.
4. Hash-based dataset identification.
5. Hash-chained audit logging with chain verification.

6. Smart city data creation and analytics dashboard.
7. Disaster monitoring and warning module.

By developing this prototype, the implementation shows that secure data-sharing procedures can actually be developed with the use of commonly used web technologies like React, Node.js, Express, and MongoDB. With the help of the dashboard analysis, controlled information regarding authorization latency, transaction cost, dataset distribution, and audit completeness can be obtained.

Moreover, this prototype proves useful for representing mechanisms for privacy and accountability within urban data-sharing architecture, even though it isn't a production blockchain framework.

7.3 LIMITATIONS

However, there exist a number of significant limitations associated with the presented research as well.

To begin with, the proposed system implementation is just a prototype and does not represent a fully functional, decentralized blockchain-based network. Although the blockchain-like logging technique was applied, and transaction cost modeling was considered during the experiment, no real experiments with distributed consensus, peer-to-peer validation, blockchain network latency, and smart contract execution time were carried out.

Furthermore, the PolicyContract element is an implementation of smart-contract behavior using backend programming techniques. Thus, the PolicyContract is not deployed on the Ethereum, Hyperledger, Polygon, or other blockchain platforms. In other words, no on-chain execution of the proposed policies can be claimed.

Additionally, although the performance metrics like gas costs and authorization latency are computed within controlled limits, they are not obtained from real blockchain-based processes but generated by the system prototype itself.

Moreover, since the smart city scenario used during the experimental analysis was synthetic and created artificially, it lacked real-world validation.

Finally, the security assessment was performed on the mechanism level only; thus, cryptographic proofs and certification procedures were not provided.

7.4 FUTURE RESEARCH DIRECTIONS

Future work should expand DataTrust_SC from simulation to actual evaluation in more realistic settings.

Firstly, an obvious research direction is the deployment of the DataTrust_SC framework in a blockchain test network. In such a setting, it will be possible to measure the cost of transactions, network latency, cost of executing smart contracts, and measure consensus behavior. At the same execution overhead, comparisons can be made between the actual and simulated results obtained.

Secondly, another direction is the actual implementation of smart contracts. While the current version of PolicyContract provides useful simulation behavior in simulations, further work should involve the development and testing of smart contracts with actual encoded access policies for correctness, cost, and security.

Thirdly, another direction should focus on improving privacy and security analyses. Future research should explore threat modeling, pen-testing, crypto-validation, and adversarial evaluations. Other advanced approaches that may be explored are Zero-Knowledge Proof-based verifications.

The fourth approach may include the application of smart city datasets or implementations within pilot projects on chosen fields such as environmental monitoring, transportation systems, utility control, or citizen services. That could help get more insights into the challenges connected with feasibility, implementation, and regulation.

Finally, further research may explore the adaptability and flexibility of access control models. For example, adaptive learning technologies could be applied for dynamically changing permissions depending on system usage, actual risk situations, or emergency cases.

Lastly, the topic of interoperability of solutions in smart cities might be investigated. Mutual data sharing by different administrative bodies would promote collaboration, scalability as well as implementation of secure privacy-preserving data exchange systems.

REFERENCES

1. Makhdoom, I., Zhou, I., Abolhasan, M., Lipman, J., & Ni, W. (2020). PrivySharing: A blockchain-based framework for privacy-preserving and secure data sharing in smart cities. *Computers & Security*, 88, 101653.
2. Zaman Khan, S., Colman, A., & H. Sarker, I. (2021, November). WikiServe: Using Wikipedia to Match IoT based Services for Situation Response. In *Proceedings of the 10th International Conference on Software and Information Engineering* (pp. 17-21).
3. Allam, Z., & Dhunny, Z. A. (2019). On big data, artificial intelligence and smart cities. *Cities*, 89, 80–91. <https://doi.org/10.1016/j.cities.2019.01.032>
4. Albino, V., Berardi, U., & Dangelico, R. M. (2015). Smart cities: Definitions, dimensions, performance, and initiatives. *Journal of Urban Technology*, 22(1), 3–21.
5. Casino, F., Dasaklis, T. K., & Patsakis, C. (2020). A systematic literature review of blockchain-based applications. *Telematics and Informatics*, 36, 55–81.
6. Chen, T., et al. (2021). Blockchain-based secure data sharing in smart cities. *Future Generation Computer Systems*.
7. Cui, J., et al. (2022). Behavioral inference attacks in smart city data. *IEEE Access*.
8. Cui, J., et al. (2023). Privacy risks in multi-source data correlation. *IEEE Internet of Things Journal*.
9. Hashem, I. A. T., et al. (2016). The role of big data in smart city. *International Journal of Information Management*, 36(5), 748–758.
10. Khan, M. A., et al. (2021). Big data analytics in smart cities. *Future Generation Computer Systems*.
11. Khan, M. A., et al. (2022). Scalable blockchain-based frameworks for smart cities. *IEEE Access*.

12. Kitchin, R. (2014). The real-time city? Big data and smart urbanism. *GeoJournal*, 79(1), 1–14.
13. Kumar, N., et al. (2021). Blockchain-based IoT security framework. *IEEE Internet of Things Journal*.
14. Li, X., et al. (2021). Blockchain-based access control mechanisms. *IEEE Access*.
15. Li, X., et al. (2022). Blockchain-enabled smart city data sharing. *Future Generation Computer Systems*.
16. Lu, Y. (2019). Blockchain and the related issues: A review. *Journal of Industrial Information Integration*, 16, 100111.
17. Minoli, D., Sohraby, K., & Occhiogrosso, B. (2021). IoT considerations for smart cities. *IEEE Communications Magazine*.
18. Mishra, P., et al. (2022). Security challenges in smart city systems. *IEEE Access*.
19. Rahman, M. A., et al. (2021). Privacy-preserving data sharing in smart cities. *IEEE Access*.
20. Rathore, M. M., et al. (2018). Urban planning and smart city decision management. *Sustainable Cities and Society*, 38, 757–766.
21. Rocher, L., Hendrickx, J. M., & de Montjoye, Y. A. (2019). Estimating the success of re-identification attacks. *Nature Communications*, 10, 3069.
22. Singh, S., et al. (2022). Blockchain-based data integrity solutions. *IEEE Access*.
23. Xu, X., et al. (2021). Blockchain-based smart contract systems. *IEEE Access*.
24. Yadav, K., et al. (2023). Trust management in smart city data sharing. *IEEE Access*.
25. Zhang, K., Ni, J., Yang, K., Liang, X., Ren, J., & Shen, X. (2018). Security and privacy in smart cities. *IEEE Communications Magazine*, 56(12), 122–128.

26. Zhang, Y., et al. (2020). Blockchain-based access control. *Future Generation Computer Systems*, 107, 555–567.
27. Zhang, Y., et al. (2023). Privacy-preserving blockchain data sharing. *Future Generation Computer Systems*.
28. Zhou, Z., et al. (2022). Secure data transmission in IoT. *IEEE Internet of Things Journal*.
29. Zheng, Z., et al. (2017). An overview of blockchain technology. *IEEE BigData Congress*, 557–564.

APPENDIX A

LIVE LINK AND SOURCE CODE REPOSITORY

THE SIMULATION PROTOTYPE DESIGNED IN THIS RESEARCH IS OPEN TO BE ACCESSED THROUGH THE GITHUB REPOSITORIES SO THAT THE DEVELOPMENT AND RESULTS ARE TRANSPARENT, REPRODUCIBLE, AND EXPANDABLE. THE SOURCE CODE USED FOR DATA GENERATION SCRIPTS, FRONTEND APPLICATION AND BACKEND SERVER IS OPEN FOR THE RESEARCHERS, DEVELOPERS, PRACTITIONERS TO REPRODUCE, TEST AND FURTHER EXPAND THE PROTOTYPE.

LIVE LINK: [HTTPS://DATA-TRUST-SC-FRONTEND.VERCEL.APP/](https://data-trust-sc-frontend.vercel.app/)

FRONTEND REPOSITORY

GITHUB: [HTTPS://GITHUB.COM/NISSANJK/DATATRUST-SC_FRONTEND_V2](https://github.com/NissanJK/DataTrust-SC_frontend_v2)

DESCRIPTION:

REACT FRONTEND FOR DATATRUST-SC, A TRUSTED AND PRIVACY-PRESERVING SMART CITY DATA DISTRIBUTION FRAMEWORK. THE APP GIVES DATA OWNERS, DATA REQUESTERS, AND SYSTEM OBSERVERS A SINGLE INTERFACE FOR UPLOADING RECORDS, REQUESTING POLICY-BASED ACCESS, MONITORING DISASTER ALERTS, VIEWING AUDIT LOGS, AND ANALYZING BLOCKCHAIN-STYLE SYSTEM METRICS.

BACKEND REPOSITORY

GITHUB: [HTTPS://GITHUB.COM/NISSANJK/DATATRUST-SC_BACKEND_V2](https://github.com/NissanJK/DataTrust-SC_backend_v2)

DESCRIPTION:

EXPRESS AND MONGODB BACKEND FOR DATATRUST-SC, A TRUSTED AND PRIVACY-PRESERVING SMART CITY DATA DISTRIBUTION FRAMEWORK. THE API STORES ENCRYPTED SMART CITY DATASETS, EVALUATES ACCESS POLICIES, RECORDS BLOCKCHAIN-STYLE AUDIT LOGS, SUPPORTS CSV IMPORT/EXPORT, AND EXPOSES DISASTER MONITORING ENDPOINTS FOR THE FRONTEND DASHBOARD.

APPENDIX B

International Islamic University Chittagong
Department of Computer Science and Engineering
Report on Final Year Thesis as Complex Engineering Problem

Title of the Project/Thesis:	DataTrust_SC: A Privacy-Preserving and Blockchain-Inspired Framework for Secure Data Sharing in Smart Cities.	
Members of the Group	Name	ID
	Jawadul Karim	C221010
	Anik Chowdhury	C201089
	Mohammed Imran	C221018
Name of the Supervisor	Mohammad Sazid Zaman Khan, Assistant Professor, Department of CSE.	

Which attributes of the Complex Engineering Problems (P) are present?

Attribute of Complex Engineering Problem	Explanation (Justify that the attribute was present and you have addressed it sufficiently)
P1: Depth of knowledge required	The project requires in-depth knowledge of cryptography (AES encryption), distributed systems principles (blockchain-inspired auditability), and web technologies (React, Node.js, MongoDB).
P2: Range of conflicting requirements	It addresses the conflict between the need for open data sharing for smart city services and the strict requirement for individual privacy and data security.
P3: Depth of analysis required (no obvious solution)	There is no obvious solution for multi-stakeholder trust in centralized systems; the project analyzes and simulates a "PolicyContract" to mimic blockchain-style deterministic authorization without full network overhead.
P4: Familiarity of issues (infrequently encountered issues)	The work deals with infrequently encountered issues like behavioral inference attacks and identity leakage in high-velocity, heterogeneous smart city data ecosystems.
P5: Extent of applicable codes	The framework aligns with abstract data protection regulations and uses industry-standard protocols like AES for encryption and SHA for hashing.
P6: Extent of stakeholder involvement and conflicting requirements	Involves a wide range of stakeholders with conflicting interests, including municipal authorities, private companies, researchers, and citizens.

P7: Interdependence	The system features high interdependence between its security layers (encryption), its authorization layer (RBAC/ABAC), and its audit layer (hash-chained logs).
----------------------------	--

Which attributes of Complex Engineering Activities (A) are involved?

Attribute of Complex Engineering Activities	Explanation (Justify that the attribute was present and you have addressed it sufficiently)
A1: Range of resources	Utilizes a diverse range of resources, including frontend frameworks, backend servers, NoSQL databases, and cryptographic libraries.
A2: Level of interactions	Involves complex interactions between data providers (IoT sensors, public agencies) and consumers (researchers, citizens) through a simulated smart contract layer.
A3: Innovation	Demonstrates innovation by applying "blockchain-inspired" principles (tamper-evident logging, deterministic policy) to solve the drawbacks of centralized smart city models.
A4: Consequences to society and the environment	The project has significant societal consequences, particularly in urban safety via the Disaster Monitoring module and protecting citizens' civil liberties through privacy-preserving data sharing.
A5: Familiarity	The activity involves non-standard engineering by simulating a blockchain environment to validate early-stage architectural feasibility for smart cities.

Comments:

The DataTrust_SC framework constitutes a solution to a complex engineering problem because it integrates high-level technical concepts—such as AES encryption and RBAC/ABAC access control—with an innovative blockchain-inspired audit trail to resolve real-world trust and privacy issues in a multi-stakeholder smart city environment. By simulating a deterministic policy enforcement model, it provides a functional prototype that balances complex requirements of data utility, system accountability, and individual privacy

Approval of Supervisor	Approval of Examination Committee
Signature	Signature